

Introduction to Computers and Python

1



Objectives

In this chapter you'll:

- Learn about exciting recent developments in computing.
- Learn computer hardware, software and Internet basics.
- Understand the data hierarchy from bits to databases.
- Understand the different types of programming languages.
- Understand object-oriented programming basics.
- Understand the strengths of Python and other leading programming languages.
- Understand the importance of libraries.
- Be introduced to key Python and data-science libraries you'll use in this book.
- Test-drive the IPython interpreter's interactive mode for executing Python code.
- Execute a Python script that animates a bar chart.
- Create and test-drive a web-browser-based Jupyter Notebook for executing Python code.
- Learn how big "big data" is and how quickly it's getting even bigger.
- Study a big-data case study on a mobile navigation app.
- Be introduced to artificial intelligence—the intersection between computer science and data science.

1.1 Introduction	1.10.2 Executing a Python Program Using the IPython Interpreter
1.2 Hardware and Software	1.10.3 Writing and Executing Code in a Jupyter Notebook
1.2.1 Moore's Law	1.11 Internet and World Wide Web
1.2.2 Computer Organization	1.11.1 Internet: A Network of Networks
1.3 Data Hierarchy	1.11.2 World Wide Web: Making the Internet User-Friendly
1.4 Machine Languages, Assembly Languages and High-Level Languages	1.11.3 The Cloud
1.5 Introduction to Object Technology	1.11.4 Internet of Things
1.6 Operating Systems	1.12 Software Technologies
1.7 Python	1.13 How Big Is Big Data?
1.8 It's the Libraries!	1.13.1 Big Data Analytics
1.8.1 Python Standard Library	1.13.2 Data Science and Big Data Are Making a Difference: Use Cases
1.8.2 Data-Science Libraries	1.14 Case Study—A Big-Data Mobile Application
1.9 Other Popular Programming Languages	1.15 Intro to Data Science: Artificial Intelligence—at the Intersection of CS and Data Science Exercises
1.10 Test-Drive: Using IPython and Jupyter Notebooks	
1.10.1 Using IPython Interactive Mode as a Calculator	

1.1 Introduction

Welcome to Python—one of the world's most widely used computer programming languages and, according to the *Popularity of Programming Languages (PYPL) Index*, the world's most popular.¹ You're probably familiar with many of the powerful tasks computers perform. In this textbook, you'll get intensive, hands-on experience writing Python instructions that command computers to perform those and other tasks. **Software** (that is, the Python instructions you write, which are also called **code**) controls **hardware** (that is, computers and related devices).

Here, we introduce terminology and concepts that lay the groundwork for the Python programming you'll learn in Chapters 2–11 and the big-data, artificial-intelligence and cloud-based case studies we present in Chapters 12–17. We'll introduce hardware and software concepts and overview the data hierarchy—from individual bits to databases, which store the massive amounts of data companies need to implement contemporary applications such as Google Search, Waze, Uber, Airbnb and a myriad of others.

We'll discuss the types of programming languages and introduce *object-oriented programming* terminology and concepts. You'll learn why Python has become so popular. We'll introduce the Python Standard Library and various data-science libraries that help you avoid “reinventing the wheel.” You'll use these libraries to create *software objects* that you'll interact with to perform significant tasks with modest numbers of instructions. We'll introduce additional software technologies that you're likely to use as you develop software.

Next, you'll work through three test-drives showing how to execute Python code:

- In the first, you'll use IPython to execute Python instructions interactively and immediately see their results.

1. <https://pyp1.github.io/PYPL.html> (as of January 2019).

- In the second, you'll execute a substantial Python application that will display an animated bar chart summarizing rolls of a six-sided die as they occur. You'll see the "Law of Large Numbers" in action. In Chapter 6, you'll build this application with the Matplotlib visualization library.
- In the last, we'll introduce Jupyter Notebooks using JupyterLab—an interactive, web-browser-based tool in which you can conveniently write and execute Python instructions. Jupyter Notebooks enable you to include text, images, audios, videos, animations and code.

In the past, most computer applications ran on "standalone" computers (that is, not networked together). Today's applications can be written with the aim of communicating among the world's computers via the Internet. We'll introduce the Internet, the World Wide Web, the Cloud and the Internet of Things (IoT), laying the groundwork for the contemporary applications you'll develop in Chapters 12–17.

You'll learn just how big "big data" is and how quickly it's getting even bigger. Next, we'll present a big-data case study on the Waze mobile navigation app, which uses many current technologies to provide dynamic driving directions that get you to your destination as quickly and as safely as possible. As we walk through those technologies, we'll mention where you'll use many of them in this book. The chapter closes with our first Intro to Data Science section in which we discuss a key intersection between computer science and data science—artificial intelligence.

1.2 Hardware and Software

Computers can perform calculations and make logical decisions phenomenally faster than human beings can. Many of today's personal computers can perform billions of calculations in one second—more than a human can perform in a lifetime. *Supercomputers* are already performing *thousands of trillions (quadrillions)* of instructions per second! IBM has developed the IBM Summit supercomputer, which can perform over 122 quadrillion calculations per second (122 *petaflops*)!² To put that in perspective, *the IBM Summit supercomputer can perform in one second almost 16 million calculations for every person on the planet!*³ And supercomputing upper limits are growing quickly.

Computers process data under the control of sequences of instructions called **computer programs** (or simply **programs**). These software programs guide the computer through ordered actions specified by people called computer **programmers**.

A computer consists of various physical devices referred to as hardware (such as the keyboard, screen, mouse, solid-state disks, hard disks, memory, DVD drives and processing units). Computing costs are *dropping dramatically*, due to rapid developments in hardware and software technologies. Computers that might have filled large rooms and cost millions of dollars decades ago are now inscribed on computer chips smaller than a fingernail, costing perhaps a few dollars each. Ironically, silicon is one of the most abundant materials on Earth—it's an ingredient in common sand. Silicon-chip technology has made computing so economical that computers have become a commodity.

2. <https://en.wikipedia.org/wiki/FLOPS>.

3. For perspective on how far computing performance has come, consider this: In his early computing days, Harvey Deitel used the Digital Equipment Corporation PDP-1 (<https://en.wikipedia.org/wiki/PDP-1>), which was capable of performing only 93,458 operations per second.

1.2.1 Moore's Law

Every year, you probably expect to pay at least a little more for most products and services. The opposite has been the case in the computer and communications fields, especially with regard to the hardware supporting these technologies. For many decades, hardware costs have fallen rapidly.

Every year or two, the capacities of computers have approximately *doubled* inexpensively. This remarkable trend often is called **Moore's Law**, named for the person who identified it in the 1960s, Gordon Moore, co-founder of Intel—one of the leading manufacturers of the processors in today's computers and embedded systems. Moore's Law *and related observations* apply especially to

- the amount of memory that computers have for programs,
- the amount of secondary storage (such as solid-state drive storage) they have to hold programs and data over longer periods of time, and
- their processor speeds—the speeds at which they *execute* their programs (that is, do their work).

Similar growth has occurred in the communications field—costs have plummeted as enormous demand for communications *bandwidth* (that is, information-carrying capacity) has attracted intense competition. We know of no other fields in which technology improves so quickly and costs fall so rapidly. Such phenomenal improvement is truly fostering the *Information Revolution*.

1.2.2 Computer Organization

Regardless of differences in *physical* appearance, computers can be envisioned as divided into various **logical units** or sections:

Input Unit

This “receiving” section obtains information (data and computer programs) from **input devices** and places it at the disposal of the other units for processing. Most user input is entered into computers through keyboards, touch screens and mouse devices. Other forms of input include receiving voice commands, scanning images and barcodes, reading from secondary storage devices (such as hard drives, Blu-ray Disc™ drives and USB flash drives—also called “thumb drives” or “memory sticks”), receiving video from a webcam and having your computer receive information from the Internet (such as when you stream videos from YouTube® or download e-books from Amazon). Newer forms of input include position data from a GPS device, motion and orientation information from an *accelerometer* (a device that responds to up/down, left/right and forward/backward acceleration) in a smartphone or wireless game controller (such as those for Microsoft® Xbox®, Nintendo Switch™ and Sony® PlayStation®) and voice input from intelligent assistants like Apple Siri®, Amazon Echo® and Google Home®.

Output Unit

This “shipping” section takes information the computer has processed and places it on various **output devices** to make it available for use outside the computer. Most information that's output from computers today is displayed on screens (including touch screens), printed on paper (“going green” discourages this), played as audio or video on smart-

phones, tablets, PCs and giant screens in sports stadiums, transmitted over the Internet or used to control other devices, such as self-driving cars, robots and “intelligent” appliances. Information is also commonly output to secondary storage devices, such as solid-state drives (SSDs), hard drives, DVD drives and USB flash drives. Popular recent forms of output are smartphone and game-controller vibration, virtual reality devices like Oculus Rift®, Sony® PlayStation® VR and Google Daydream View™ and Samsung Gear VR®, and mixed reality devices like Magic Leap® One and Microsoft HoloLens™.

Memory Unit

This rapid-access, relatively low-capacity “warehouse” section retains information that has been entered through the input unit, making it immediately available for processing when needed. The memory unit also retains processed information until it can be placed on output devices by the output unit. Information in the memory unit is *volatile*—it’s typically lost when the computer’s power is turned off. The memory unit is often called either **memory**, **primary memory** or **RAM** (Random Access Memory). Main memories on desktop and notebook computers contain as much as 128 GB of RAM, though 8 to 16 GB is most common. GB stands for gigabytes; a gigabyte is approximately one billion bytes. A **byte** is eight bits. A bit is either a 0 or a 1.

Arithmetic and Logic Unit (ALU)

This “manufacturing” section performs *calculations*, such as addition, subtraction, multiplication and division. It also contains the *decision* mechanisms that allow the computer, for example, to compare two items from the memory unit to determine whether they’re equal. In today’s systems, the ALU is part of the next logical unit, the CPU.

Central Processing Unit (CPU)

This “administrative” section coordinates and supervises the operation of the other sections. The CPU tells the input unit when information should be read into the memory unit, tells the ALU when information from the memory unit should be used in calculations and tells the output unit when to send information from the memory unit to specific output devices. Most computers have **multicore processors** that implement multiple processors on a single integrated-circuit chip. Such processors can perform many operations simultaneously. A *dual-core processor* has two CPUs, a *quad-core processor* has four and an *octa-core processor* has eight. Intel has some processors with up to 72 cores. Today’s desktop computers have processors that can execute billions of instructions per second.

Secondary Storage Unit

This is the long-term, high-capacity “warehousing” section. Programs or data not actively being used by the other units normally are placed on secondary storage devices (e.g., your *hard drive*) until they’re again needed, possibly hours, days, months or even years later. Information on secondary storage devices is *persistent*—it’s preserved even when the computer’s power is turned off. Secondary storage information takes much longer to access than information in primary memory, but its cost per unit is much less. Examples of secondary storage devices include solid-state drives (SSDs), hard drives, read/write Blu-ray drives and USB flash drives. Many current drives hold terabytes (TB) of data—a **terabyte** is approximately one trillion bytes). Typical hard drives on desktop and notebook computers hold up to 4 TB, and some recent desktop-computer hard drives hold up to 15 TB.⁴

✓ Self Check for Section 1.2

1 (Fill-In) Every year or two, the capacities of computers have approximately doubled inexpensively. This remarkable trend often is called _____.

Answer: Moore's Law.

2 (True/False) Information in the memory unit is *persistent*—it's preserved even when the computer's power is turned off

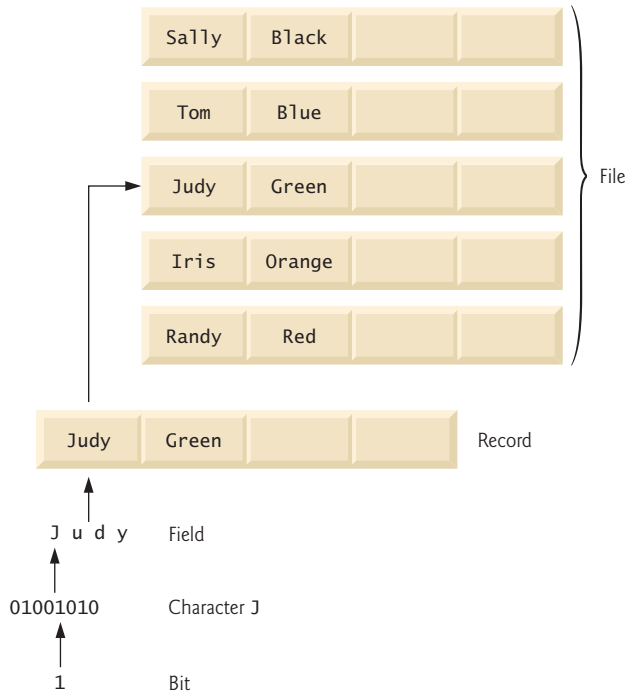
Answer: False. Information in the memory unit is *volatile*—it's typically lost when the computer's power is turned off.

3 (Fill-In) Most computers have _____ processors that implement multiple processors on a single integrated-circuit chip. Such processors can perform many operations simultaneously.

Answer: multicore.

1.3 Data Hierarchy

Data items processed by computers form a **data hierarchy** that becomes larger and more complex in structure as we progress from the simplest data items (called “bits”) to richer ones, such as characters and fields. The following diagram illustrates a portion of the data hierarchy:



4. <https://www.zdnet.com/article/worlds-biggest-hard-drive-meet-western-digitals-15tb-monster/>.

Bits

A **bit** (short for “binary digit”—a digit that can assume one of *two* values) is the smallest data item in a computer. It can have the value 0 or 1. Remarkably, the impressive functions performed by computers involve only the simplest manipulations of 0s and 1s—*examining a bit’s value*, *setting a bit’s value* and *reversing a bit’s value* (from 1 to 0 or from 0 to 1). Bits form the basis of the binary number system, which you can study in-depth in our online “Number Systems” appendix.

Characters

Work with data in the low-level form of bits is tedious. Instead, people prefer to work with *decimal digits* (0–9), *letters* (A–Z and a–z) and *special symbols* such as

\$ @ % & * () - + " : ; , ? /

Digits, letters and special symbols are known as **characters**. The computer’s **character set** contains the characters used to write programs and represent data items. Computers process only 1s and 0s, so a computer’s character set represents every character as a pattern of 1s and 0s. Python uses **Unicode**[®] characters that are composed of one, two, three or four bytes (8, 16, 24 or 32 bits, respectively)—known as **UTF-8 encoding**.⁵

Unicode contains characters for many of the world’s languages. The **ASCII (American Standard Code for Information Interchange)** character set is a subset of Unicode that represents letters (a–z and A–Z), digits and some common special characters. You can view the ASCII subset of Unicode at

<https://www.unicode.org/charts/PDF/U0000.pdf>

The Unicode charts for all languages, symbols, emojis and more are viewable at

<http://www.unicode.org/charts/>

Fields

Just as characters are composed of bits, **fields** are composed of characters or bytes. A field is a group of characters or bytes that conveys meaning. For example, a field consisting of uppercase and lowercase letters can be used to represent a person’s name, and a field consisting of decimal digits could represent a person’s age.

Records

Several related fields can be used to compose a **record**. In a payroll system, for example, the record for an employee might consist of the following fields (possible types for these fields are shown in parentheses):

- Employee identification number (a whole number).
- Name (a string of characters).
- Address (a string of characters).
- Hourly pay rate (a number with a decimal point).
- Year-to-date earnings (a number with a decimal point).
- Amount of taxes withheld (a number with a decimal point).

5. <https://docs.python.org/3/howto/unicode.html>.

Thus, a record is a group of related fields. All the fields listed above belong to the same employee. A company might have many employees and a payroll record for each.

Files

A **file** is a group of related records. More generally, a file contains arbitrary data in arbitrary formats. In some operating systems, a file is viewed simply as a *sequence of bytes*—any organization of the bytes in a file, such as organizing the data into records, is a view created by the application programmer. You'll see how to do that in Chapter 9, “Files and Exceptions.” It's not unusual for an organization to have many files, some containing billions, or even trillions, of characters of information.

Databases

A **database** is a collection of data organized for easy access and manipulation. The most popular model is the *relational database*, in which data is stored in simple *tables*. A table includes *records* and *fields*. For example, a table of students might include first name, last name, major, year, student ID number and grade-point-average fields. The data for each student is a record, and the individual pieces of information in each record are the fields. You can *search*, *sort* and otherwise manipulate the data, based on its relationship to multiple tables or databases. For example, a university might use data from the student database in combination with data from databases of courses, on-campus housing, meal plans, etc. We discuss databases in Chapter 17, “Big Data: Hadoop, Spark, NoSQL and IoT.”

Big Data

The table below shows some common byte measurements:

Unit	Bytes	Which is approximately
1 kilobyte (KB)	1024 bytes	10^3 (1024) bytes exactly
1 megabyte (MB)	1024 kilobytes	10^6 (1,000,000) bytes
1 gigabyte (GB)	1024 megabytes	10^9 (1,000,000,000) bytes
1 terabyte (TB)	1024 gigabytes	10^{12} (1,000,000,000,000) bytes
1 petabyte (PB)	1024 terabytes	10^{15} (1,000,000,000,000,000) bytes
1 exabyte (EB)	1024 petabytes	10^{18} (1,000,000,000,000,000,000) bytes
1 zettabyte (ZB)	1024 exabytes	10^{21} (1,000,000,000,000,000,000,000) bytes

The amount of data being produced worldwide is enormous and its growth is accelerating. **Big data** applications deal with massive amounts of data. This field is growing quickly, creating lots of opportunity for software developers. Millions of IT jobs globally already are supporting big data applications. Section 1.13 discusses big data in more depth. You'll study big data and associated technologies in Chapter 17.



Self Check

I (Fill-In) A(n) _____ (short for “binary digit”—a digit that can assume one of two values) is the smallest data item in a computer.

Answer: bit.

2 (True/False) In some operating systems, a file is viewed simply as a sequence of bytes—any organization of the bytes in a file, such as organizing the data into records, is a view created by the application programmer.

Answer: True.

3 (Fill-In) A database is a collection of data organized for easy access and manipulation. The most popular model is the _____ database, in which data is stored in simple tables.

Answer: *relational*

1.4 Machine Languages, Assembly Languages and High-Level Languages

Programmers write instructions in various programming languages, some directly understandable by computers and others requiring intermediate *translation* steps. Hundreds of such languages are in use today. These may be divided into three general types:

1. Machine languages.
2. Assembly languages.
3. High-level languages.

Machine Languages

Any computer can directly understand only its own **machine language**, defined by its hardware design. Machine languages generally consist of strings of numbers (ultimately reduced to 1s and 0s) that instruct computers to perform their most elementary operations one at a time. Machine languages are *machine dependent* (a particular machine language can be used on only one type of computer). Such languages are cumbersome for humans. For example, here's a section of an early machine-language payroll program that adds overtime pay to base pay and stores the result in gross pay:

```
+1300042774
+1400593419
+1200274027
```

Assembly Languages and Assemblers

Programming in machine language was simply too slow and tedious for most programmers. Instead of using the strings of numbers that computers could directly understand, programmers began using English-like abbreviations to represent elementary operations. These abbreviations formed the basis of **assembly languages**. *Translator programs* called **assemblers** were developed to convert assembly-language programs to machine language at computer speeds. The following section of an assembly-language payroll program also adds overtime pay to base pay and stores the result in gross pay:

```
load    basepay
add     overpay
store   grosspay
```

Although such code is clearer to humans, it's incomprehensible to computers until translated to machine language.

High-Level Languages and Compilers

With the advent of assembly languages, computer usage increased rapidly, but programmers still had to use numerous instructions to accomplish even the simplest tasks. To speed the programming process, **high-level languages** were developed in which single statements could be written to accomplish substantial tasks. A typical high-level-language program contains many statements, known as the program's **source code**.

Translator programs called **compilers** convert high-level-language source code into machine language. High-level languages allow you to write instructions that look almost like everyday English and contain commonly used mathematical notations. A payroll program written in a high-level language might contain a *single* statement such as

```
grossPay = basePay + overTimePay
```

From the programmer's standpoint, high-level languages are preferable to machine and assembly languages. Python is among the world's most widely used high-level programming languages.

Interpreters

Compiling a large high-level language program into machine language can take considerable computer time. *Interpreter* programs, developed to execute high-level language programs directly, avoid the delay of compilation, although they run slower than compiled programs. The most widely used Python implementation—CPython (which is written in the C programming language)—uses a clever mixture of compilation and interpretation to run programs.⁶



Self Check

1 (Fill-In) Translator programs called _____ convert assembly-language programs to machine language at computer speeds.

Answer: assemblers.

2 (Fill-In) _____ programs, developed to execute high-level-language programs directly, avoid the delay of compilation, although they run slower than compiled programs

Answer: Interpreter.

3 (True/False) High-level languages allow you to write instructions that look almost like everyday English and contain commonly used mathematical notations.

Answer: True.

1.5 Introduction to Object Technology

As demands for new and more powerful software are soaring, building software quickly, correctly and economically is important. *Objects*, or more precisely, the *classes* objects come from, are essentially *reusable* software components. There are date objects, time objects, audio objects, video objects, automobile objects, people objects, etc. Almost any *noun* can be reasonably represented as a software object in terms of *attributes* (e.g., name, color and size) and *behaviors* (e.g., calculating, moving and communicating). Software-development groups can use a modular, object-oriented design-and-implementation approach to be

6. <https://opensource.com/article/18/4/introduction-python-bytecode>.

much more productive than with earlier popular techniques like “structured programming.” Object-oriented programs are often easier to understand, correct and modify.

Automobile as an Object

To help you understand objects and their contents, let’s begin with a simple analogy. Suppose you want to *drive a car and make it go faster by pressing its accelerator pedal*. What must happen before you can do this? Well, before you can drive a car, someone has to *design* it. A car typically begins as engineering drawings, similar to the *blueprints* that describe the design of a house. These drawings include the design for an accelerator pedal. The pedal *hides* from the driver the complex mechanisms that make the car go faster, just as the brake pedal “hides” the mechanisms that slow the car, and the steering wheel “hides” the mechanisms that turn the car. This enables people with little or no knowledge of how engines, braking and steering mechanisms work to drive a car easily.

Just as you cannot cook meals in the blueprint of a kitchen, you cannot drive a car’s engineering drawings. Before you can drive a car, it must be *built* from the engineering drawings that describe it. A completed car has an *actual* accelerator pedal to make it go faster, but even that’s not enough—the car won’t accelerate on its own (hopefully!), so the driver must *press* the pedal to accelerate the car.

Methods and Classes

Let’s use our car example to introduce some key object-oriented programming concepts. Performing a task in a program requires a **method**. The method houses the program statements that perform its tasks. The method hides these statements from its user, just as the accelerator pedal of a car hides from the driver the mechanisms of making the car go faster. In Python, a program unit called a **class** houses the set of methods that perform the class’s tasks. For example, a class that represents a bank account might contain one method to *deposit* money to an account, another to *withdraw* money from an account and a third to *inquire* what the account’s balance is. A class is similar in concept to a car’s engineering drawings, which house the design of an accelerator pedal, steering wheel, and so on.

Instantiation

Just as someone has to *build a car* from its engineering drawings before you can drive a car, you must *build an object* of a class before a program can perform the tasks that the class’s methods define. The process of doing this is called *instantiation*. An object is then referred to as an **instance** of its class.

Reuse

Just as a car’s engineering drawings can be *reused* many times to build many cars, you can *reuse* a class many times to build many objects. Reuse of existing classes when building new classes and programs saves time and effort. Reuse also helps you build more reliable and effective systems because existing classes and components often have undergone extensive *testing*, *debugging* (that is, finding and removing errors) and *performance tuning*. Just as the notion of *interchangeable parts* was crucial to the Industrial Revolution, reusable classes are crucial to the software revolution that has been spurred by object technology.

In Python, you’ll typically use a *building-block approach* to create your programs. To avoid reinventing the wheel, you’ll use existing high-quality pieces wherever possible. This software reuse is a key benefit of object-oriented programming.

Messages and Method Calls

When you drive a car, pressing its gas pedal sends a *message* to the car to perform a task—that is, to go faster. Similarly, you *send messages to an object*. Each message is implemented as a **method call** that tells a method of the object to perform its task. For example, a program might call a bank-account object’s *deposit* method to increase the account’s balance.

Attributes and Instance Variables

A car, besides having capabilities to accomplish tasks, also has *attributes*, such as its color, its number of doors, the amount of gas in its tank, its current speed and its record of total miles driven (i.e., its odometer reading). Like its capabilities, the car’s attributes are represented as part of its design in its engineering diagrams (which, for example, include an odometer and a fuel gauge). As you drive an actual car, these attributes are carried along with the car. Every car maintains its *own* attributes. For example, each car knows how much gas is in its own gas tank, but *not* how much is in the tanks of *other* cars.

An object, similarly, has attributes that it carries along as it’s used in a program. These attributes are specified as part of the object’s class. For example, a bank-account object has a *balance attribute* that represents the amount of money in the account. Each bank-account object knows the balance in the account it represents, but *not* the balances of the *other* accounts in the bank. Attributes are specified by the class’s **instance variables**. A class’s (and its object’s) attributes and methods are intimately related, so classes wrap together their attributes and methods.

Inheritance

A new class of objects can be created conveniently by **inheritance**—the new class (called the **subclass**) starts with the characteristics of an existing class (called the **superclass**), possibly customizing them and adding unique characteristics of its own. In our car analogy, an object of class “convertible” certainly *is an* object of the more *general* class “automobile,” but more *specifically*, the roof can be raised or lowered.

Object-Oriented Analysis and Design (OOAD)

Soon you’ll be writing programs in Python. How will you create the **code** (i.e., the program instructions) for your programs? Perhaps, like many programmers, you’ll simply turn on your computer and start typing. This approach may work for small programs (like the ones we present in the early chapters of the book), but what if you were asked to create a software system to control thousands of automated teller machines for a major bank? Or suppose you were asked to work on a team of 1,000 software developers building the next generation of the U.S. air traffic control system? For projects so large and complex, you should not simply sit down and start writing programs.

To create the best solutions, you should follow a detailed **analysis** process for determining your project’s **requirements** (i.e., defining *what* the system is supposed to do), then develop a **design** that satisfies them (i.e., specifying *how* the system should do it). Ideally, you’d go through this process and carefully review the design (and have your design reviewed by other software professionals) before writing any code. If this process involves analyzing and designing your system from an object-oriented point of view, it’s called an **object-oriented analysis-and-design (OOAD) process**. Languages like Python are object-oriented. Programming in such a language, called **object-oriented programming (OOP)**, allows you to implement an object-oriented design as a working system.

✓ Self Check for Section 1.5

1 (Fill-In) To create the best solutions, you should follow a detailed analysis process for determining your project's _____ (i.e., defining *what* the system is supposed to do) and developing a design that satisfies them (i.e., specifying *how* the system should do it).

Answer: requirements.

2 (Fill-In) The size, shape, color and weight of an object are _____ of the object's class.

Answer: attributes.

3 (True/False) Objects, or more precisely, the classes objects come from, are essentially reusable software components.

Answer: True.

1.6 Operating Systems

Operating systems are software systems that make using computers more convenient for users, application developers and system administrators. They provide services that allow each application to execute safely, efficiently and *concurrently* with other applications. The software that contains the core components of the operating system is called the **kernel**. Linux, Windows and macOS are popular desktop computer operating systems—you can use any of these with this book. The most popular mobile operating systems used in smartphones and tablets are Google's Android and Apple's iOS.

Windows—A Proprietary Operating System

In the mid-1980s, Microsoft developed the **Windows operating system**, consisting of a graphical user interface built on top of DOS (Disk Operating System)—an enormously popular personal-computer operating system that users interacted with by typing commands. Windows 10 is Microsoft's latest operating system—it includes the Cortana personal assistant for voice interactions. Windows is a *proprietary* operating system—it's controlled by Microsoft exclusively. Windows is by far the world's most widely used desktop operating system.

Linux—An Open-Source Operating System

The **Linux operating system** is among the greatest successes of the *open-source* movement. **Open-source software** departs from the *proprietary* software development style that dominated software's early years. With open-source development, individuals and companies *contribute* their efforts in developing, maintaining and evolving software in exchange for the right to use that software for their own purposes, typically at *no charge*. Open-source code is often scrutinized by a much larger audience than proprietary software, so errors often get removed faster. Open source also encourages innovation.

There are many organizations in the open-source community. Some key ones are:

- **Python Software Foundation** (responsible for Python).
- **GitHub** (provides tools for managing open-source projects—it has millions of them under development).
- The **Apache Software Foundation** (originally the creators of the Apache web server, they now oversee 350 open-source projects, including several big data infrastructure technologies we present in Chapter 17).

- The **Eclipse Foundation** (the Eclipse Integrated Development Environment helps programmers conveniently develop software)
- The **Mozilla Foundation** (creators of the Firefox web browser)
- **OpenML** (which focuses on open-source tools and data for machine learning—you'll explore machine learning in Chapter 15).
- **OpenAI** (which does research on artificial intelligence and publishes open-source tools used in AI reinforcement-learning research).
- **OpenCV** (which focuses on open-source computer-vision tools that can be used across a range of operating systems and programming languages—you'll study computer-vision applications in Chapter 16).

Rapid improvements to computing and communications, decreasing costs and open-source software have made it much easier and more economical to create software-based businesses now than just a decade ago. A great example is Facebook, which was launched from a college dorm room and built with open-source software.

The **Linux kernel** is the core of the most popular open-source, freely distributed, full-featured operating system. It's developed by a loosely organized team of volunteers and is popular in servers, personal computers and embedded systems (such as the computer systems at the heart of smartphones, smart TVs and automobile systems). Unlike that of proprietary operating systems like Microsoft's Windows and Apple's macOS, Linux source code (the program code) is available to the public for examination and modification and is free to download and install. As a result, Linux users benefit from a huge community of developers actively debugging and improving the kernel, and the ability to customize the operating system to meet specific needs.

Apple's macOS and Apple's iOS for iPhone® and iPad® Devices

Apple, founded in 1976 by Steve Jobs and Steve Wozniak, quickly became a leader in personal computing. In 1979, Jobs and several Apple employees visited Xerox PARC (Palo Alto Research Center) to learn about Xerox's desktop computer that featured a graphical user interface (GUI). That GUI served as the inspiration for the Apple Macintosh, launched in 1984.

The Objective-C programming language, created by Stepstone in the early 1980s, added capabilities for object-oriented programming (OOP) to the C programming language. Steve Jobs left Apple in 1985 and founded NeXT Inc. In 1988, NeXT licensed Objective-C from Stepstone and developed an Objective-C compiler and libraries which were used as the platform for the NeXTSTEP operating system's user interface, and Interface Builder—used to construct graphical user interfaces.

Jobs returned to Apple in 1996 when they bought NeXT. Apple's **macOS operating system** is a descendant of NeXTSTEP. Apple's proprietary operating system, **iOS**, is derived from macOS and is used in the iPhone, iPad, Apple Watch and Apple TV devices. In 2014, Apple introduced its new Swift programming language, which became open source in 2015. The iOS app-development community has largely shifted from Objective-C to Swift.

Google's Android

Android—the fastest growing mobile and smartphone operating system—is based on the Linux kernel and the Java programming language. Android is open source and free.

According to idc.com, as of 2018, Android had 86.8% of the global smartphone market share, compared to 13.2% for Apple.⁷ The Android operating system is used in numerous smartphones, e-reader devices, tablets, in-store touch-screen kiosks, cars, robots, multimedia players and more.

Billions of Devices

In use today are Billions of personal computers and an even larger number of mobile devices. The following table lists many computerized devices. The explosive growth of mobile phones, tablets and other devices is creating significant opportunities for programming mobile apps. There are now various tools that enable you to use Python for Android and iOS app development, including BeeWare, Kivy, PyMob, Pythonista and others. Many are **cross-platform**, meaning that you can use them to develop apps that will run portably on Android, iOS and other platforms (like the web).

Computerized devices

Access control systems	Airplane systems	ATMs
Automobiles	Blu-ray Disc™ players	Building controls
Cable boxes	Copiers	Credit cards
CT scanners	Desktop computers	e-Readers
Game consoles	GPS navigation systems	Home appliances
Home security systems	Internet-of-Things gateways	Light switches
Logic controllers	Lottery systems	Medical devices
Mobile phones	MRIs	Network switches
Optical sensors	Parking meters	Personal computers
Point-of-sale terminals	Printers	Robots
Routers	Servers	Smartcards
Smart meters	Smartpens	Smartphones
Tablets	Televisions	Thermostats
Transportation passes	TV set-top boxes	Vehicle diagnostic systems



Self Check for Section I.6

1 (Fill-In) Windows is a(n) _____ operating system—it's controlled by Microsoft exclusively.

Answer: proprietary.

2 (True/False) Proprietary code is often scrutinized by a much larger audience than open-source software, so errors often get removed faster.

Answer: False. Open-source code is often scrutinized by a much larger audience than proprietary software, so errors often get removed faster.

3 (True/False) iOS dominates the global smartphone market over Android.

Answer: False. Android currently controls 88% of the smartphone market.

7. <https://www.idc.com/promo/smartphone-market-share/os>.

1.7 Python

Python is an object-oriented scripting language that was released publicly in 1991. It was developed by Guido van Rossum of the National Research Institute for Mathematics and Computer Science in Amsterdam.

Python has rapidly become one of the world's most popular programming languages. It's now particularly popular for educational and scientific computing,⁸ and it recently surpassed the programming language R as the most popular data-science programming language.^{9,10,11} Here are some reasons why Python is popular and everyone should consider learning it:^{12,13,14}

- It's open source, free and widely available with a massive open-source community.
- It's easier to learn than languages like C, C++, C# and Java, enabling novices and professional developers to get up to speed quickly.
- It's easier to read than many other popular programming languages.
- It's widely used in education.¹⁵
- It enhances developer productivity with extensive standard libraries and *thousands* of third-party open-source libraries, so programmers can write code faster and perform complex tasks with minimal code. We'll say more about this in Section 1.8.
- There are massive numbers of free open-source Python applications.
- It's popular in web development (e.g., Django, Flask).
- It supports popular programming paradigms—procedural, functional, object-oriented and reflective.¹⁶ We'll begin introducing functional-style programming features in Chapter 4 and use them in subsequent chapters.
- It simplifies concurrent programming—with `asyncio` and `async/await`, you're able to write single-threaded concurrent code¹⁷, greatly simplifying the inherently complex processes of writing, debugging and maintaining that code.¹⁸
- There are lots of capabilities for enhancing Python performance.
- It's used to build anything from simple scripts to complex apps with massive numbers of users, such as Dropbox, YouTube, Reddit, Instagram and Quora.¹⁹

8. <https://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017>.

9. <https://www.kdnuggets.com/2017/08/python-overtakes-r-leader-analytics-data-science.html>.

10. <https://www.r-bloggers.com/data-science-job-report-2017-r-passes-sas-but-python-leaves-them-both-behind/>.

11. <https://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017>.

12. <https://dbader.org/blog/why-learn-python>.

13. <https://simpleprogrammer.com/2017/01/18/7-reasons-why-you-should-learn-python/>.

14. <https://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017>.

15. Tollervey, N., *Python in Education: Teach, Learn, Program* (O'Reilly Media, Inc., 2015).

16. [https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language)).

17. <https://docs.python.org/3/library/asyncio.html>.

18. <https://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017>.

19. https://www.hartmannsoftware.com/Blog/Articles_from_Software_Fans/Most-Famous-Software-Programs-Written-in-Python.

- It's popular in artificial intelligence, which is enjoying explosive growth, in part because of its special relationship with data science.
- It's widely used in the financial community.²⁰
- There's an extensive job market for Python programmers across many disciplines, especially in data-science-oriented positions, and Python jobs are among the highest paid of all programming jobs.^{21,22}

Anaconda Python Distribution

We use the Anaconda Python distribution because it's easy to install on Windows, macOS and Linux and supports the latest versions of Python (3.7 at the time of this writing), the IPython interpreter (introduced in Section 1.10.1) and Jupyter Notebooks (introduced in Section 1.10.3). Anaconda also includes other software packages and libraries commonly used in Python programming and data science, allowing students to focus on learning Python, computer science and data science, rather than software installation issues. The IPython interpreter²³ has features that help students and professionals explore, discover and experiment with Python, the Python Standard Library and the extensive set of third-party libraries.

Zen of Python

We adhere to Tim Peters' *The Zen of Python*, which summarizes Python creator Guido van Rossum's design principles for the language. This list can be viewed in IPython with the command `import this`. The Zen of Python is defined in Python Enhancement Proposal (PEP) 20. "A PEP is a design document providing information to the Python community, or describing a new feature for Python or its processes or environment."²⁴



Self Check

1 (Fill-In) The _____ summarizes Python creator Guido van Rossum's design principles for the Python language.

Answer: Zen of Python.

2 (True/False) The Python language supports popular programming paradigms—procedural, functional, object-oriented and reflective.

Answer: True.

3 (True/False) R is most the popular data-science programming language.

Answer: False. Python recently surpassed R as the most popular data-science programming language.

20. Kolanovic, M. and R. Krishnamachari, *Big Data and AI Strategies: Machine Learning and Alternative Data Approach to Investing* (J.P. Morgan, 2017).

21. <https://www.infoworld.com/article/3170838/developer/get-paid-10-programming-languages-to-learn-in-2017.html>.

22. <https://medium.com/@ChallengeRocket/top-10-of-programming-languages-with-the-highest-salaries-in-2017-4390f468256e>.

23. <https://ipython.org/>.

24. <https://www.python.org/dev/peps/pep-0001/>.

1.8 It's the Libraries!

Throughout the book, we focus on using existing libraries to help you avoid “reinventing the wheel,” thus leveraging your program-development efforts. Often, rather than developing lots of original code—a costly and time-consuming process—you can simply create an object of a pre-existing library class, which takes only a single Python statement. So, libraries will help you perform significant tasks with modest amounts of code. You'll use a broad range of Python standard libraries, data-science libraries and other third-party libraries.

1.8.1 Python Standard Library

The **Python Standard Library** provides rich capabilities for text/binary data processing, mathematics, functional-style programming, file/directory access, data persistence, data compression/archiving, cryptography, operating-system services, concurrent programming, interprocess communication, networking protocols, JSON/XML/other Internet data formats, multimedia, internationalization, GUI, debugging, profiling and more. The following table lists some of the Python Standard Library modules that we use in examples or that you'll explore in the exercises.

Some of the Python Standard Library modules we use in the book

collections —Additional data structures beyond lists, tuples, dictionaries and sets.	os —Interacting with the operating system.
csv —Processing comma-separated value files.	timeit —Performance analysis.
datetime, time —Date and time manipulations.	queue —First-in, first-out data structure.
decimal —Fixed-point and floating-point arithmetic, including monetary calculations.	random —Pseudorandom numbers.
doctest —Simple unit testing via validation tests and expected results embedded in docstrings.	re —Regular expressions for pattern matching.
json —JavaScript Object Notation (JSON) processing for use with web services and NoSQL document databases.	sqlite3 —SQLite relational database access.
math —Common math constants and operations.	statistics —Mathematical statistics functions like mean, median, mode and variance.
	string —String processing.
	sys —Command-line argument processing; standard input, standard output and standard error streams.

1.8.2 Data-Science Libraries

Python has an enormous and rapidly growing community of open-source developers in many fields. One of the biggest reasons for Python's popularity is the extraordinary range of open-source libraries developed by the open-source community. One of our goals is to create examples, exercises, projects (EEPs) and implementation case studies that give you an engaging, challenging and entertaining introduction to Python programming, while also involving you in hands-on data science, key data-science libraries and more. You'll be amazed at the substantial tasks you can accomplish in just a few lines of code. The following table lists various popular data-science libraries. You'll use many of these as you work through our data-science examples, exercises and projects. For visualization, we'll focus primarily on Matplotlib and Seaborn, but there are many more. For a nice summary of Python visualization libraries see <http://pyviz.org/>.

Popular Python libraries used in data science

Scientific Computing and Statistics

NumPy (Numerical Python)—Python does not have a built-in array data structure. It uses lists, which are convenient but relatively slow. NumPy provides the more efficient `ndarray` data structure to represent lists and matrices, and it also provides routines for processing such data structures.

SciPy (Scientific Python)—Built on NumPy, SciPy adds routines for scientific processing, such as integrals, differential equations, additional matrix processing and more. `scipy.org` controls SciPy and NumPy.

StatsModels—Provides support for estimations of statistical models, statistical tests and statistical data exploration.

Data Manipulation and Analysis

Pandas—An extremely popular library for data manipulations. Pandas makes abundant use of NumPy's `ndarray`. Its two key data structures are `Series` (one dimensional) and `DataFrames` (two dimensional).

Visualization

Matplotlib—A highly customizable visualization and plotting library. Supported plots include regular, scatter, bar, contour, pie, quiver, grid, polar axis, 3D and text.

Seaborn—A higher-level visualization library built on Matplotlib. Seaborn adds a nicer look-and-feel, additional visualizations and enables you to create visualizations with less code.

Machine Learning, Deep Learning and Reinforcement Learning

scikit-learn—Top machine-learning library. Machine learning is a subset of AI. Deep learning is a subset of machine learning that focuses on neural networks.

Keras—One of the easiest to use deep-learning libraries. Keras runs on top of TensorFlow (Google), CNTK (Microsoft's cognitive toolkit for deep learning) or Theano (Université de Montréal).

TensorFlow—From Google, this is the most widely used deep learning library. TensorFlow works with GPUs (graphics processing units) or Google's custom TPUs (Tensor processing units) for performance. TensorFlow is important in AI and big data analytics—where processing demands are enormous. You'll use the version of Keras that's built into TensorFlow.

OpenAI Gym—A library and environment for developing, testing and comparing reinforcement-learning algorithms. You'll explore this in the Chapter 16 exercises.

Natural Language Processing (NLP)

NLTK (Natural Language Toolkit)—Used for natural language processing (NLP) tasks.

TextBlob—An object-oriented NLP text-processing library built on the NLTK and pattern NLP libraries. TextBlob simplifies many NLP tasks.

Gensim—Similar to NLTK. Commonly used to build an index for a collection of documents, then determine how similar another document is to each of those in the index. You'll explore this in the Chapter 12 exercises.



Self Check for Section 1.8

1 (Fill-In) _____ help you avoid “reinventing the wheel,” thus leveraging your program-development efforts.

Answer: Libraries.

2 (Fill-In) The _____ provides rich capabilities for many common Python programming tasks.

Answer: Python Standard Library.

1.9 Other Popular Programming Languages

The following is a brief introduction to several other popular programming languages—in the next section, we take a deeper look at Python:

- *Basic* was developed in the 1960s at Dartmouth College to familiarize novices with programming techniques. Many of its latest versions are object-oriented.
- *C* was developed in the early 1970s by Dennis Ritchie at Bell Laboratories. It initially became widely known as the UNIX operating system’s development language. Today, most code for general-purpose operating systems and other performance-critical systems is written in C or C++.
- C++, which is based on C, was developed by Bjarne Stroustrup in the early 1980s at Bell Laboratories. C++ provides features that enhance the C language and adds capabilities for object-oriented programming.
- *Java*—Sun Microsystems in 1991 funded an internal corporate research project led by James Gosling, which resulted in the C++-based object-oriented programming language called Java. A key goal of Java is to enable developers to write programs that will run on a great variety of computer systems. This is called “write once, run anywhere.” Java is used to develop enterprise applications, to enhance the functionality of web servers (the computers that provide the content to our web browsers), to provide applications for consumer devices (e.g., smartphones, tablets, television set-top boxes, appliances, automobiles and more) and for many other purposes. Java was originally the key language for developing Android smartphone and tablet apps, though several other languages are now supported.
- *C#* (based on C++ and Java) is one of Microsoft’s three primary object-oriented programming languages—the other two are Visual C++ and Visual Basic. C# was developed to integrate the web into computer applications and is now widely used to develop many types of applications. As part of Microsoft’s many open-source initiatives implemented over the last few years, they now offer open-source versions of C# and Visual Basic.
- *JavaScript* is the most widely used scripting language. It’s primarily used to add programmability to web pages—for example, animations and interactivity with the user. All major web browsers support it. Many Python visualization libraries output JavaScript as part of visualizations that you can interact with in your web browser. Tools like NodeJS also enable JavaScript to run outside of web browsers.
- *Swift*, which was introduced in 2014, is Apple’s programming language for developing iOS and macOS apps. Swift is a contemporary language that includes popular features from languages such as Objective-C, Java, C#, Ruby, Python and others. Swift is open source, so it can be used on non-Apple platforms as well.
- *R* is a popular open-source programming language for statistical applications and visualization. Python and R are the two most widely data-science languages.

✓ Self Check

1 (Fill-In) Today, most code for general-purpose operating systems and other performance-critical systems is written in _____.

Answer: C or C++.

2 (Fill-In) A key goal of _____ is to enable developers to write programs that will run on a great variety of computer systems and computer-controlled devices. This is sometimes called “write once, run anywhere.”

Answer: Java.

1.10 Test-Drive: Using IPython and Jupyter Notebooks

In this section, you’ll test-drive the IPython interpreter²⁵ in two modes:

- In **interactive mode**, you’ll enter small bits of Python code called **snippets** and immediately see their results.
- In **script mode**, you’ll execute code loaded from a file that has the `.py` extension (short for Python). Such files are called **scripts** or **programs**, and they’re generally longer than the code snippets you’ll do in interactive mode.

Then, you’ll learn how to use the browser-based environment known as the Jupyter Notebook for writing and executing Python code.²⁶

1.10.1 Using IPython Interactive Mode as a Calculator

Let’s use IPython interactive mode to evaluate simple arithmetic expressions.

Entering IPython in Interactive Mode

First, open a command-line window on your system:

- On macOS, open a **Terminal** from the **Applications** folder’s **Utilities** subfolder.
- On Windows, open the **Anaconda Command Prompt** from the start menu.
- On Linux, open your system’s **Terminal** or shell (this varies by Linux distribution).

In the command-line window, type `ipython`, then press *Enter* (or *Return*). You’ll see text like the following, this varies by platform and by IPython version:

```
Python 3.7.0 | packaged by conda-forge | (default, Jan 20 2019, 17:24:52)
Type 'copyright', 'credits' or 'license' for more information
IPython 6.5.0 -- An enhanced Interactive Python. Type '?' for help.
```

```
In [1]:
```

The text “In [1]:” is a *prompt*, indicating that IPython is waiting for your input. You can type `?` for help or begin entering snippets, as you’ll do momentarily.

25. Before reading this section, follow the instructions in the Before You Begin section to install the Anaconda Python distribution, which contains the IPython interpreter.

26. Jupyter supports many programming languages by installing their “kernels.” For more information see <https://github.com/jupyter/jupyter/wiki/Jupyter-kernels>.

Evaluating Expressions

In interactive mode, you can evaluate expressions:

```
In [1]: 45 + 72
Out[1]: 117
```

```
In [2]:
```

After you type `45 + 72` and press *Enter*, IPython *reads* the snippet, *evaluates* it and *prints* its result in `Out[1]`.²⁷ Then IPython displays the `In [2]` prompt to show that it's waiting for you to enter your second snippet. For each new snippet, IPython adds 1 to the number in the square brackets. Each `In [1]` prompt in the book indicates that we've started a new interactive session. We generally do that for each new section of a chapter.

Let's evaluate a more complex expression:

```
In [2]: 5 * (12.7 - 4) / 2
Out[2]: 21.75
```

Python uses the asterisk (*) for multiplication and the forward slash (/) for division. As in mathematics, parentheses force the evaluation order, so the parenthesized expression `(12.7 - 4)` evaluates first, giving 8.7. Next, `5 * 8.7` evaluates giving 43.5. Then, `43.5 / 2` evaluates, giving the result 21.75, which IPython displays in `Out[2]`. Whole numbers, like 5, 4 and 2, are called **integers**. Numbers with decimal points, like 12.7, 43.5 and 21.75, are called **floating-point numbers**.

Exiting Interactive Mode

To leave interactive mode, you can:

- Type the `exit` command at the current `In []` prompt and press *Enter* to exit immediately.
- Type the key sequence `<Ctrl> + d` (or `<control> + d`). This displays the prompt "Do you really want to exit ([y]/n)?". The square brackets around `y` indicate that it's the default response—pressing *Enter* submits the default response and exits.
- Type `<Ctrl> + d` (or `<control> + d`) twice (macOS and Linux only).



Self Check

1 (Fill-In) In IPython interactive mode, you'll enter small bits of Python code called _____ and immediately see their results.

Answer: snippets.

2 In IPython _____ mode, you'll execute Python code loaded from a file that has the `.py` extension (short for Python).

Answer: script.

3 (IPython Session) Evaluate the expression `5 * (3 + 4)` both with and without the parentheses. Do you get the same result? Why or why not?

27. In the next chapter, you'll see that there are some cases in which `Out[]` is not displayed.

Answer: You get different results because snippet [1] first calculates $3 + 4$, which is 7, then multiplies that by 5. Snippet [2] first multiplies $5 * 3$, which is 15, then adds that to 4.

```
In [1]: 5 * (3 + 4)
Out[1]: 35
```

```
In [2]: 5 * 3 + 4
Out[2]: 19
```

1.10.2 Executing a Python Program Using the IPython Interpreter

In this section, you'll execute a script named `RollDieDynamic.py` that you'll write in Chapter 6. The **.py extension** indicates that the file contains Python source code. The script `RollDieDynamic.py` simulates rolling a six-sided die. It presents a colorful animated visualization that dynamically graphs the frequencies of each die face.

Changing to This Chapter's Examples Folder

You'll find the script in the book's `ch01` source-code folder. In the Before You Begin section you extracted the `examples` folder to your user account's Documents folder. Each chapter has a folder containing that chapter's source code. The folder is named `ch##`, where `##` is a two-digit chapter number from 01 to 17. First, open your system's command-line window. Next, use the `cd` ("change directory") command to change to the `ch01` folder:

- On macOS/Linux, type `cd ~/Documents/examples/ch01`, then press *Enter*.
- On Windows, type `cd C:\Users\YourAccount\Documents\examples\ch01`, then press *Enter*.

Executing the Script

To execute the script, type the following command at the command line, then press *Enter*:

```
ipython RollDieDynamic.py 6000 1
```

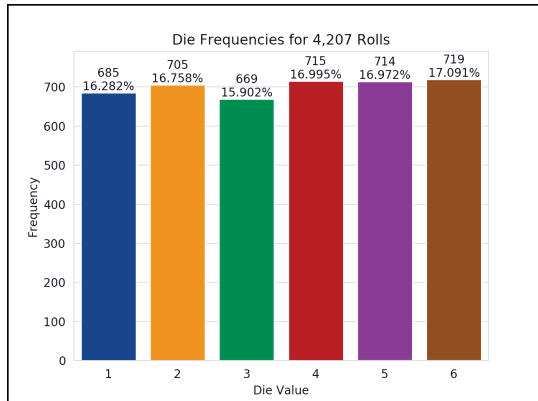
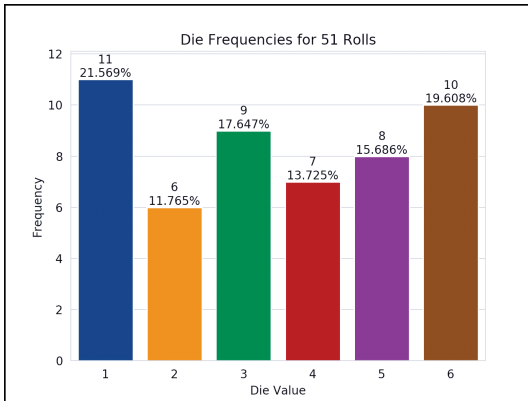
The script displays a window, showing the visualization. The numbers 6000 and 1 tell this script the number of times to roll dice and how many dice to roll each time. In this case, we'll update the chart 6000 times for 1 die at a time.

For a six-sided die, the values 1 through 6 should each occur with "equal likelihood"—the probability of each is $1/6^{\text{th}}$ or about 16.667%. If we roll a die 6000 times, we'd expect about 1000 of each face. Like coin tossing, die rolling is *random*, so there could be some faces with fewer than 1000, some with 1000 and some with more than 1000. We took the screen captures on the next page during the script's execution. This script uses randomly generated die values, so your results will differ. Experiment with the script by changing the value 1 to 100, 1000 and 10000. Notice that as the number of die rolls gets larger, the frequencies zero in on 16.667%. This is a phenomenon of the "Law of Large Numbers."

Creating Scripts

Typically, you create your Python source code in an editor that enables you to type text. Using the editor, you type a program, make any necessary corrections and save it to your computer. **Integrated development environments (IDEs)** provide tools that support the entire software-development process, such as editors, debuggers for locating **logic errors** that cause programs to execute incorrectly and more. Some popular Python IDEs include Spyder (which comes with Anaconda), PyCharm and Visual Studio Code.

Roll the dice 6000 times and roll 1 die each time:
`!python RollDieDynamic.py 6000 1`



Problems That May Occur at Execution Time

Programs often do not work on the first try. For example, an executing program might try to divide by zero (an illegal operation in Python). This would cause the program to display an error message. If this occurred in a script, you'd return to the editor, make the necessary corrections and re-execute the script to determine whether the corrections fixed the problem(s).

Errors such as division by zero occur as a program runs, so they're called **runtime errors** or **execution-time errors**. **Fatal runtime errors** cause programs to terminate immediately without having successfully performed their jobs. **Non-fatal runtime errors** allow programs to run to completion, often producing incorrect results.



Self Check

1 (Discussion) When the example in this section finishes all 6000 rolls, does the chart show that the die faces appeared *about* 1000 times each?

Answer: Most likely, yes. This example is based on random-number generation, so the results may vary. Because of this randomness, most of the counts will be a little more than 1000 or a little less.

2 (Discussion) Run the example in this section again. Do the faces appear the same number of times as they did in the previous execution?

Answer: Probably not. This example uses random-number generation, so successive executions likely will produce different results. In Chapter 4, we'll show how to force Python to produce the same sequence of random numbers. This is important for *reproducibility*—a crucial data-science topic you'll investigate in the chapter exercises and throughout the book. You'll want other data scientists to be able to reproduce your results. Also, you'll want to be able to reproduce your own experimental results. This is helpful when you find and fix an error in your program and want to make sure that you've corrected it properly.

1.10.3 Writing and Executing Code in a Jupyter Notebook

The Anaconda Python Distribution that you installed in the Before You Begin section comes with the **Jupyter Notebook**—an interactive, browser-based environment in which

you can write and execute code and intermix the code with text, images and video. Jupyter Notebooks are broadly used in the data-science community in particular and the broader scientific community in general. They're the preferred means of doing Python-based data analytics studies and *reproducibly* communicating their results. The Jupyter Notebook environment actually supports many programming languages.

For your convenience, all of the book's source code also is provided in Jupyter Notebooks that you can simply load and execute. In this section, you'll use the **JupyterLab** interface, which enables you to manage your notebook files and other files that your notebooks use (like images and videos). As you'll see, JupyterLab also makes it convenient to write code, execute it, see the results, modify the code and execute it again.

You'll see that coding in a Jupyter Notebook is similar to working with IPython—in fact, Jupyter Notebooks use IPython by default. In this section, you'll create a notebook, add the code from Section 1.10.1 to it and execute that code.

Opening JupyterLab in Your Browser

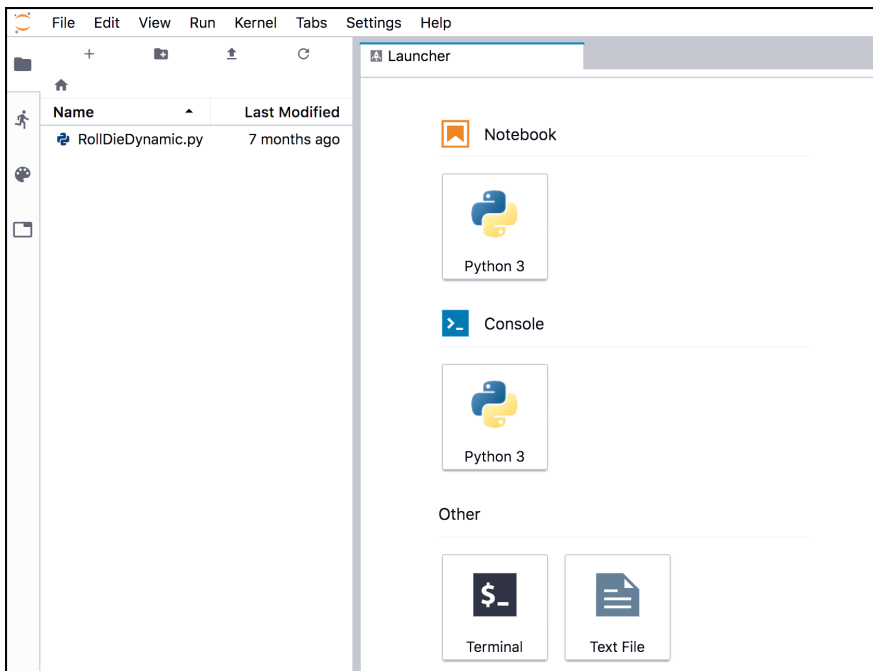
To open JupyterLab, change to the `ch01` examples folder in your Terminal, shell or Anaconda Command Prompt (as in Section 1.10.2), type the following command, then press *Enter* (or *Return*):

```
jupyter lab
```

This executes the Jupyter Notebook server on your computer and opens JupyterLab in your default web browser, showing the `ch01` folder's contents in the **File Browser** tab



at the left side of the JupyterLab interface:



The Jupyter Notebooks server enables you to load and run Jupyter Notebooks in your web browser. From the JupyterLab **Files** tab, you can double-click files to open them in the right side of the window where the **Launcher** tab is currently displayed. Each file you open appears as a separate tab in this part of the window. If you accidentally close your browser, you can reopen JupyterLab by entering the following address in your web browser

`http://localhost:8888/lab`

Creating a New Jupyter Notebook

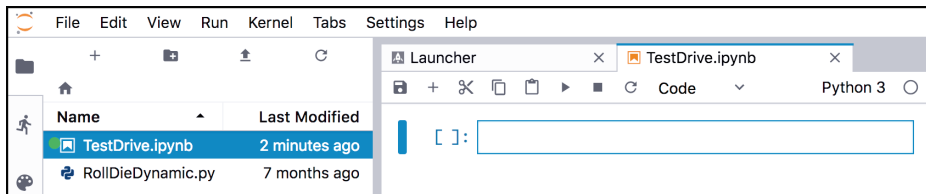
In the **Launcher** tab under **Notebook**, click the **Python 3** button to create a new Jupyter Notebook named `Untitled.ipynb` in which you can enter and execute Python 3 code. The file extension `.ipynb` is short for IPython Notebook—the original name of the Jupyter Notebook.

Renaming the Notebook

Rename `Untitled.ipynb` as `TestDrive.ipynb`:

1. Right-click the `Untitled.ipynb` tab and select **Rename Notebook....**
2. Change the name to `TestDrive.ipynb` and click **RENAME**.

The top of JupyterLab should now appear as follows:

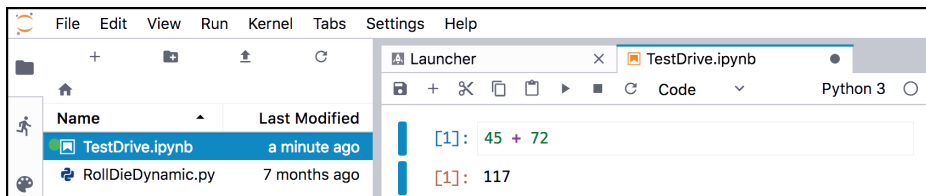


Evaluating an Expression

The unit of work in a notebook is a **cell** in which you can enter code snippets. By default, a new notebook contains one cell—the rectangle in the `TestDrive.ipynb` notebook—but you can add more. To the cell's left, the notation `[ ]:` is where the Jupyter Notebook will display the cell's snippet number *after* you execute the cell. Click in the cell, then type the expression

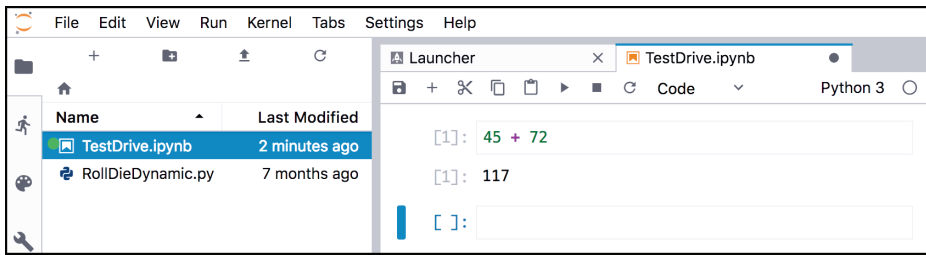
`45 + 72`

To execute the current cell's code, type *Ctrl + Enter* (or *control + Enter*). JupyterLab executes the code in IPython, then displays the results below the cell:



Adding and Executing Another Cell

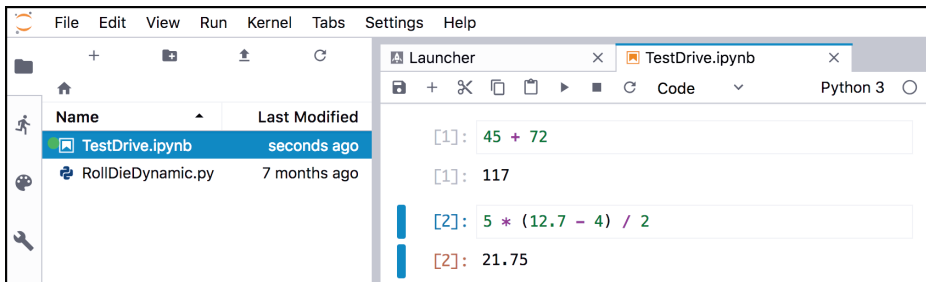
Let's evaluate a more complex expression. First, click the **+** button in the toolbar above the notebook's first cell—this adds a new cell below the current one:



Click in the new cell, then type the expression

$$5 * (12.7 - 4) / 2$$

and execute the cell by typing *Ctrl + Enter* (or *control + Enter*):



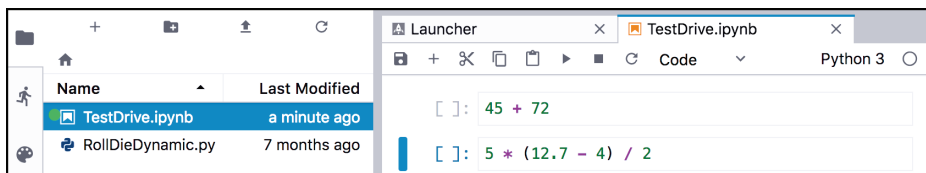
Saving the Notebook

If your notebook has unsaved changes, the **X** in the notebook's tab will change to ●. To save the notebook, select the **File** menu in JupyterLab (not at the top of your browser's window), then select **Save Notebook**.

Notebooks Provided with Each Chapter's Examples

For your convenience, each chapter's examples also are provided as ready-to-execute notebooks without their outputs. This enables you to work through them snippet-by-snippet and see the outputs appear as you execute each snippet.

So that we can show you how to load an existing notebook and execute its cells, let's reset the `TestDrive.ipynb` notebook to remove its output and snippet numbers. This will return it to a state like the notebooks we provide for the subsequent chapters' examples. From the **Kernel** menu select **Restart Kernel and Clear All Outputs...**, then click the **RESTART** button. The preceding command also is helpful whenever you wish to re-execute a notebook's snippets. The notebook should now appear as follows:



From the **File** menu, select **Save Notebook**, then click the TestDrive.ipynb tab's **X** button to close the notebook.

Opening and Executing an Existing Notebook

When you launch JupyterLab from a given chapter's examples folder, you'll be able to open notebooks from that folder or any of its subfolders. Once you locate a specific notebook, double-click it to open it. Open the TestDrive.ipynb notebook again now. Once a notebook is open, you can execute each cell individually, as you did earlier in this section, or you can execute the entire notebook at once. To do so, from the **Run** menu select **Run All Cells**. The notebook will execute the cells in order, displaying each cell's output below that cell.

Closing JupyterLab

When you're done with JupyterLab, you can close its browser tab, then in the Terminal, shell or Anaconda Command Prompt from which you launched JupyterLab, type *Ctrl + c* (or *control + c*) twice.

JupyterLab Tips

While working in JupyterLab, you might find these tips helpful:

- If you need to enter and execute many snippets, you can execute the current cell *and* add a new one below it by typing *Shift + Enter*, rather than *Ctrl + Enter* (or *control + Enter*).
- As you get into the later chapters, some of the snippets you'll enter in Jupyter Notebooks will contain many lines of code. To display line numbers within each cell, select **Show line numbers** from JupyterLab's **View** menu.

More Information on Working with JupyterLab

JupyterLab has many more features that you'll find helpful. We recommend that you read the Jupyter team's introduction to JupyterLab at:

<https://jupyterlab.readthedocs.io/en/stable/index.html>

For a quick overview, click **Overview** under **GETTING STARTED**. Also, under **USER GUIDE** read the introductions to **The JupyterLab Interface**, **Working with Files**, **Text Editor** and **Notebooks** for many additional features.



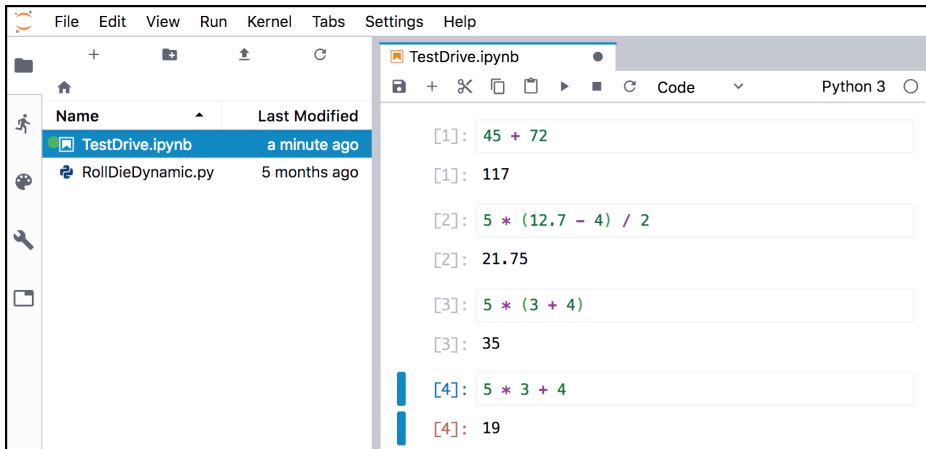
Self Check

1 (*True/False*) Jupyter Notebooks are the preferred means of doing Python-based data analytics studies and reproducibly communicating their results.

Answer: True.

2 (*Jupyter Notebook Session*) Ensure that JupyterLab is running, then open your TestDrive.ipynb notebook. Add and execute two more snippets that evaluate the expression $5 * (3 + 4)$ both with and without the parentheses. You should see the same results as in Section 1.10.1's Self Check Exercise 3.

Answer:



1.11 Internet and World Wide Web

In the late 1960s, ARPA—the Advanced Research Projects Agency of the United States Department of Defense—rolled out plans for networking the main computer systems of approximately a dozen ARPA-funded universities and research institutions. The computers were to be connected with communications lines operating at speeds on the order of 50,000 bits per second, a stunning rate at a time when most people (of the few who even had networking access) were connecting over telephone lines to computers at a rate of 110 bits per second. Academic research was about to take a giant leap forward. ARPA proceeded to implement what quickly became known as the ARPANET, the precursor to today's **Internet**. Today's fastest Internet speeds are on the order of billions of bits per second with trillion-bits-per-second (terabit) speeds already being tested!²⁸

Things worked out differently from the original plan. Although the ARPANET enabled researchers to network their computers, its main benefit proved to be the capability for quick and easy communication via what came to be known as electronic mail (e-mail). This is true even on today's Internet, with e-mail, instant messaging, file transfer and social media such as Snapchat, Instagram, Facebook and Twitter enabling billions of people worldwide to communicate quickly and easily.

The protocol (set of rules) for communicating over the ARPANET became known as the **Transmission Control Protocol (TCP)**. TCP ensured that messages, consisting of sequentially numbered pieces called *packets*, were properly delivered from sender to receiver, arrived intact and were assembled in the correct order.

1.11.1 Internet: A Network of Networks

In parallel with the early evolution of the Internet, organizations worldwide were implementing their own networks for both intra-organization (that is, within an organization) and inter-organization (that is, between organizations) communication. A huge variety of networking hardware and software appeared. One challenge was to enable these different

28. <https://testinternetspeed.org/blog/bt-testing-1-4-terabit-internet-connections/>.

networks to communicate with each other. ARPA accomplished this by developing the **Internet Protocol (IP)**, which created a true “network of networks,” the current architecture of the Internet. The combined set of protocols is now called **TCP/IP**. Each Internet-connected device has an **IP address**—a unique numerical identifier used by devices communicating via TCP/IP to locate one another on the Internet.

Businesses rapidly realized that by using the Internet, they could improve their operations and offer new and better services to their clients. Companies started spending large amounts of money to develop and enhance their Internet presence. This generated fierce competition among communications carriers and hardware and software suppliers to meet the increased infrastructure demand. As a result, **bandwidth**—the information-carrying capacity of communications lines—on the Internet has increased tremendously, while hardware costs have plummeted.

1.1.1.2 World Wide Web: Making the Internet User-Friendly

The **World Wide Web** (simply called “the web”) is a collection of hardware and software associated with the Internet that allows computer users to locate and view documents (with various combinations of text, graphics, animations, audios and videos) on almost any subject. In 1989, Tim Berners-Lee of CERN (the European Organization for Nuclear Research) began developing **HyperText Markup Language (HTML)**—the technology for sharing information via “hyperlinked” text documents. He also wrote communication protocols such as **HyperText Transfer Protocol (HTTP)** to form the backbone of his new hypertext information system, which he referred to as the World Wide Web.

In 1994, Berners-Lee founded the **World Wide Web Consortium (W3C)**, <https://www.w3.org>), devoted to developing web technologies. One of the W3C’s primary goals is to make the web universally accessible to everyone regardless of disabilities, language or culture.

1.1.1.3 The Cloud

More and more computing today is done “in the cloud”—that is, distributed across the Internet worldwide. The apps you use daily are heavily dependent on various **cloud-based services** that use massive clusters of computing resources (computers, processors, memory, disk drives, etc.) and databases that communicate over the Internet with each other and the apps you use. A service that provides access to itself over the Internet is known as a **web service**. As you’ll see, using cloud-based services in Python often is as simple as creating a software object and interacting with it. That object then uses web services that connect to the cloud on your behalf.

Throughout the Chapters 12–17 examples and exercises, you’ll work with many cloud-based services:

- In Chapters 13 and 17, you’ll use Twitter’s web services (via the Python library Tweepy) to get information about specific Twitter users, search for tweets from the last seven days and to receive streams of tweets as they occur—that is, in **real time**.
- In Chapters 12 and 13, you’ll use the Python library TextBlob to translate text between languages. Behind the scenes, TextBlob uses the Google Translate web service to perform those translations.

- In Chapter 14, you'll use the IBM Watson's Text to Speech, Speech to Text and Translate services. You'll implement a traveler's assistant translation app that enables you to speak a question in English, transcribes the speech to text, translates the text to Spanish and speaks the Spanish text. The app then allows you to speak a Spanish response (in case you don't speak Spanish, we provide an audio file you can use), transcribes the speech to text, translates the text to English and speaks the English response. Via IBM Watson demos, you'll also experiment with many other Watson cloud-based services in Chapter 14.
- In Chapter 17, you'll work with Microsoft Azure's HDInsight service and other Azure web services as you learn to implement big-data applications using Apache Hadoop and Spark. Azure is Microsoft's set of cloud-based services.
- In Chapter 17, you'll use the Dweet.io web service to simulate an Internet-connected thermostat that publishes temperature readings online. You'll also use a web-based service to create a "dashboard" that visualizes the temperature readings over time and warns you if the temperature gets too low or too high.
- In Chapter 17, you'll use a web-based dashboard to visualize a simulated stream of live sensor data from the PubNub web service. You'll also create a Python app that visualizes a PubNub simulated stream of live stock-price changes.
- In multiple exercises, you'll research, explore and use Wikipedia web services.

In most cases, you'll create Python objects that interact with web services on your behalf, hiding the details of how to access these services over the Internet.

Mashups

The applications-development methodology of *mashups* enables you to rapidly develop powerful software applications by combining (often free) complementary web services and other forms of information feeds—as you'll do in our IBM Watson traveler's assistant translation app. One of the first mashups combined the real-estate listings provided by <http://www.craigslist.org> with the mapping capabilities of Google Maps to offer maps that showed the locations of homes for sale or rent in a given area.

ProgrammableWeb (<http://www.programmableweb.com/>) provides a directory of over 20,750 web services and almost 8,000 mashups. They also provide how-to guides and sample code for working with web services and creating your own mashups. According to their website, some of the most widely used web services are Facebook, Google Maps, Twitter and YouTube.

1.1.1.4 Internet of Things

The Internet is no longer just a network of *computers*—it's an **Internet of Things (IoT)**. A *thing* is any object with an IP address and the ability to send, and in some cases receive, data automatically over the Internet. Such *things* include:

- a car with a transponder for paying tolls,
- monitors for parking-space availability in a garage,
- a heart monitor implanted in a human,
- water quality monitors,

- a smart meter that reports energy usage,
- radiation detectors,
- item trackers in a warehouse,
- mobile apps that can track your movement and location,
- smart thermostats that adjust room temperatures based on weather forecasts and activity in the home, and
- intelligent home appliances.

According to [statista.com](https://www.statista.com/statistics/471264/iot-number-of-connected-devices-world-wide/), there are already over 23 billion IoT devices in use today, and there could be over 75 billion IoT devices in 2025.²⁹



Self Check for Section 1.11

1 (Fill-In) The _____ was the precursor to today's Internet.

Answer: ARPANET.

2 (Fill-In) The _____ (simply called “the web”) is a collection of hardware and software associated with the Internet that allows computer users to locate and view documents (with various combinations of text, graphics, animations, audios and videos).

Answer: World Wide Web.

3 (Fill-In) In the Internet of Things (IoT), a *thing* is any object with a(n) _____ and the ability to send, and in some cases receive, data automatically over the Internet.

Answer: IP address.

1.12 Software Technologies

As you learn about and work in software development, you'll frequently encounter the following buzzwords:

- **Refactoring:** Reworking programs to make them clearer and easier to maintain while preserving their correctness and functionality. Many IDEs contain built-in *refactoring tools* to do major portions of the reworking automatically.
- **Design patterns:** Proven architectures for constructing flexible and maintainable object-oriented software. The field of design patterns tries to enumerate those recurring patterns, encouraging software designers to *reuse* them to develop better-quality software using less time, money and effort.
- **Cloud computing:** You can use software and data stored in the “cloud”—i.e., accessed on remote computers (or servers) via the Internet and available on demand—rather than having it stored locally on your desktop, notebook computer or mobile device. This allows you to increase or decrease computing resources to meet your needs at any given time, which is more cost effective than purchasing hardware to provide enough storage and processing power to meet occasional peak demands. Cloud computing also saves money by shifting to the

29. <https://www.statista.com/statistics/471264/iot-number-of-connected-devices-world-wide/>.

service provider the burden of managing these apps (such as installing and upgrading the software, security, backups and disaster recovery).

- **Software Development Kits (SDKs)**—The tools and documentation that developers use to program applications. For example, in Chapter 14, you'll use the Watson Developer Cloud Python SDK to interact with IBM Watson services from a Python application.



Self Check

I (*Fill-In*) _____ is the process of reworking programs to make them clearer and easier to maintain while preserving their correctness and functionality.

Answer: refactoring.

I.13 How Big Is Big Data?

For computer scientists and data scientists, data is now as important as writing programs. According to IBM, approximately 2.5 quintillion bytes (2.5 *exabytes*) of data are created daily,³⁰ and 90% of the world's data was created in the last two years.³¹ According to IDC, the global data supply will reach 175 *zettabytes* (equal to 175 trillion gigabytes or 175 billion terabytes) annually by 2025.³² Consider the following examples of various popular data measures.

Megabytes (MB)

One megabyte is about one million (actually 2^{20}) bytes. Many of the files we use on a daily basis require one or more MBs of storage. Some examples include:

- MP3 audio files—High-quality MP3s range from 1 to 2.4 MB per minute.³³
- Photos—JPEG format photos taken on a digital camera can require about 8 to 10 MB per photo.
- Video—Smartphone cameras can record video at various resolutions. Each minute of video can require many megabytes of storage. For example, on one of our iPhones, the **Camera** settings app reports that 1080p video at 30 frames-per-second (FPS) requires 130 MB/minute and 4K video at 30 FPS requires 350 MB/minute.

Gigabytes (GB)

One gigabyte is about 1000 megabytes (actually 2^{30} bytes). A dual-layer DVD can store up to 8.5 GB³⁴, which translates to:

- as much as 141 hours of MP3 audio,
- approximately 1000 photos from a 16-megapixel camera,

30. <https://www.ibm.com/blogs/watson/2016/06/welcome-to-the-world-of-a-i/>.

31. <https://public.dhe.ibm.com/common/ssi/ecm/wr/en/wr112345usen/watson-customer-engagement-watson-marketing-wr-other-papers-and-reports-wr112345usen-20170719.pdf>.

32. <https://www.networkworld.com/article/3325397/storage/idc-expect-175-zettabytes-of-data-worldwide-by-2025.html>.

33. <https://www.audiomountain.com/tech/audio-file-size.html>.

34. <https://en.wikipedia.org/wiki/DVD>.

- approximately 7.7 minutes of 1080p video at 30 FPS, or
- approximately 2.85 minutes of 4K video at 30 FPS.

The current highest-capacity Ultra HD Blu-ray discs can store up to 100 GB of video.³⁵ Streaming a 4K movie can use between 7 and 10 GB per hour (highly compressed).

Terabytes (TB)

One terabyte is about 1000 gigabytes (actually 2^{40} bytes). Recent disk drives for desktop computers come in sizes up to 15 TB,³⁶ which is equivalent to:

- approximately 28 years of MP3 audio,
- approximately 1.68 million photos from a 16-megapixel camera,
- approximately 226 hours of 1080p video at 30 FPS and
- approximately 84 hours of 4K video at 30 FPS.

Nimbus Data now has the largest solid-state drive (SSD) at 100 TB, which can store 6.67 times the 15-TB examples of audio, photos and video listed above.³⁷

Petabytes, Exabytes and Zettabytes

There are nearly four billion people online creating about 2.5 quintillion bytes of data each day³⁸—that's 2500 petabytes (each petabyte is about 1000 terabytes) or 2.5 exabytes (each exabyte is about 1000 petabytes). According to a March 2016 *AnalyticsWeek* article, within five years there will be over 50 billion devices connected to the Internet (most of them through the Internet of Things, which we discuss in Sections 1.11.4 and 17.8) and by 2020 we'll be producing 1.7 megabytes of new data every second *for every person on the planet*.³⁹ At today's numbers (approximately 7.7 billion people⁴⁰), that's about

- 13 petabytes of new data per second,
- 780 petabytes per minute,
- 46,800 petabytes (46.8 exabytes) per hour and
- 1,123 exabytes per day—that's 1.123 zettabytes (ZB) per day (each zettabyte is about 1000 exabytes).

That's the equivalent of over 5.5 million hours (over 600 years) of 4K video every day or approximately 116 billion photos every day!

Additional Big-Data Stats

For a real-time sense of big data, check out <https://www.internetlivestats.com>, with various statistics, including the numbers so far today of

- Google searches.

35. https://en.wikipedia.org/wiki/Ultra_HD_Blu-ray.

36. <https://www.zdnet.com/article/worlds-biggest-hard-drive-meet-western-digital-s-15tb-monster/>.

37. <https://www.cinema5d.com/nimbus-data-100tb-ssd-worlds-largest-ssd/>.

38. <https://public.dhe.ibm.com/common/ssi/ecm/wr/en/wr112345usen/watson-customer-engagement-watson-marketing-wr-other-papers-and-reports-wr112345usen-20170719.pdf>.

39. <https://analyticsweek.com/content/big-data-facts/>.

40. https://en.wikipedia.org/wiki/World_population.

- Tweets.
- Videos viewed on YouTube.
- Photos uploaded on Instagram.

You can click each statistic to drill down for more information. For instance, they say over 250 *billion* tweets have been sent in 2018.

Some other interesting big-data facts:

- Every hour, YouTube users upload 24,000 hours of video, and almost 1 billion hours of video are watched on YouTube every day.⁴¹
- Every second, there are 51,773 GBs (or 51.773 TBs) of Internet traffic, 7894 tweets sent, 64,332 Google searches and 72,029 YouTube videos viewed.⁴²
- On Facebook each day there are 800 million “likes,”⁴³ 60 million emojis are sent,⁴⁴ and there are over two billion searches of the more than 2.5 trillion Facebook posts since the site’s inception.⁴⁵
- In June 2017, Will Marshall, CEO of Planet, said the company has 142 satellites that image the whole planet’s land mass once per day. They add one million images and seven TBs of new data each day. Together with their partners, they’re using machine learning on that data to improve crop yields, see how many ships are in a given port and track deforestation. With respect to Amazon deforestation, he said: “Used to be we’d wake up after a few years and there’s a big hole in the Amazon. Now we can literally count every tree on the planet every day.”⁴⁶

Domo, Inc. has a nice infographic called “Data Never Sleeps 6.0” showing how much data is generated *every minute*, including:⁴⁷

- 473,400 tweets sent.
- 2,083,333 Snapchat photos shared.
- 97,222 hours of Netflix video viewed.
- 12,986,111 million text messages sent.
- 49,380 Instagram posts.
- 176,220 Skype calls.
- 750,000 Spotify songs streamed.
- 3,877,140 Google searches.
- 4,333,560 YouTube videos watched.

41. <https://www.brandwatch.com/blog/youtube-stats/>.

42. <http://www.internetlivestats.com/one-second>.

43. <https://newsroom.fb.com/news/2017/06/two-billion-people-coming-together-on-facebook>.

44. <https://mashable.com/2017/07/17/facebook-world-emoji-day/>.

45. <https://techcrunch.com/2016/07/27/facebook-will-make-you-talk/>.

46. <https://www.bloomberg.com/news/videos/2017-06-30/learning-from-planet-s-shoe-boxed-sized-satellites-video>, June 30, 2017.

47. <https://www.domo.com/learn/data-never-sleeps-6>.

Computing Power Over the Years

Data is getting more massive and so is the computing power for processing it. The performance of today's processors is often measured in terms of **FLOPS (floating-point operations per second)**. In the early to mid-1990s, the fastest supercomputer speeds were measured in gigaflops (10^9 FLOPS). By the late 1990s, Intel produced the first teraflop (10^{12} FLOPS) supercomputers. In the early-to-mid 2000s, speeds reached hundreds of teraflops, then in 2008, IBM released the first petaflop (10^{15} FLOPS) supercomputer. Currently, the fastest supercomputer—the IBM Summit, located at the Department of Energy's (DOE) Oak Ridge National Laboratory (ORNL)—is capable of 122.3 petaflops.⁴⁸

Distributed computing can link thousands of personal computers via the Internet to produce even more FLOPS. In late 2016, the Folding@home network—a distributed network in which people volunteer their personal computers' resources for use in disease research and drug design⁴⁹—was capable of over 100 petaflops.⁵⁰ Companies like IBM are now working toward supercomputers capable of exaflops (10^{18} FLOPS).⁵¹

The **quantum computers** now under development theoretically could operate at 18,000,000,000,000,000,000 times the speed of today's "conventional computers!"⁵² This number is so extraordinary that in one second, a quantum computer theoretically could do staggeringly more calculations than the total that have been done by all computers since the world's first computer appeared. This almost unimaginable computing power could wreak havoc with blockchain-based cryptocurrencies like Bitcoin. Engineers are already rethinking blockchain to prepare for such massive increases in computing power.⁵³

The history of supercomputing power is that it eventually works its way down from research labs, where extraordinary amounts of money have been spent to achieve those performance numbers, into "reasonably priced" commercial computer systems and even desktop computers, laptops, tablets and smartphones.

Computing power's cost continues to decline, especially with cloud computing. People used to ask the question, "How much computing power do I need on my system to deal with my *peak* processing needs?" Today, that thinking has shifted to "Can I quickly carve out on the cloud what I need *temporarily* for my most demanding computing chores?" You pay for only what you use to accomplish a given task.

Processing the World's Data Requires Lots of Electricity

Data from the world's Internet-connected devices is exploding, and processing that data requires tremendous amounts of energy. According to a recent article, energy use for processing data in 2015 was growing at 20% per year and consuming approximately three to five percent of the world's power. The article says that total data-processing power consumption could reach 20% by 2025.⁵⁴

48. <https://en.wikipedia.org/wiki/FLOPS>.

49. <https://en.wikipedia.org/wiki/Folding@home>.

50. <https://en.wikipedia.org/wiki/FLOPS>.

51. <https://www.ibm.com/blogs/research/2017/06/supercomputing-weather-model-exascale/>.

52. <https://medium.com/@n.biedrzycki/only-god-can-count-that-fast-the-world-of-quantum-computing-406a0a91fcf4>.

53. <https://singularityhub.com/2017/11/05/is-quantum-computing-an-existential-threat-to-blockchain-technology/>.

54. <https://www.theguardian.com/environment/2017/dec/11/tsunami-of-data-could-consume-fifth-global-electricity-by-2025>.

Another enormous electricity consumer is the blockchain-based cryptocurrency Bitcoin. Processing just one Bitcoin transaction uses approximately the same amount of energy as powering the average American home for a week. The energy use comes from the process Bitcoin “miners” use to prove that transaction data is valid.⁵⁵

According to some estimates, a year of Bitcoin transactions consumes more energy than many countries.⁵⁶ Together, Bitcoin and Ethereum (another popular blockchain-based platform and cryptocurrency) consume more energy per year than Israel and almost as much as Greece.⁵⁷

Morgan Stanley predicted in 2018 that “the electricity consumption required to create cryptocurrencies this year could actually outpace the firm’s projected global electric vehicle demand—in 2025.”⁵⁸ This situation is unsustainable, especially given the huge interest in blockchain-based applications, even beyond the cryptocurrency explosion. The blockchain community is working on fixes.^{59,60}

Big-Data Opportunities

The big-data explosion is likely to continue exponentially for years to come. With 50 billion computing devices on the horizon, we can only imagine how many more there will be over the next few decades. It’s crucial for businesses, governments, the military, and even individuals to get a handle on all this data.

It’s interesting that some of the best writings about big data, data science, artificial intelligence and more are coming out of distinguished business organizations, such as J.P. Morgan, McKinsey and more. Big data’s appeal to big business is undeniable given the rapidly accelerating accomplishments. Many companies are making significant investments and getting valuable results through technologies in this book, such as big data, machine learning, deep learning, and natural-language processing. This is forcing competitors to invest as well, rapidly increasing the need for computing professionals with data-science and computer science experience. This growth is likely to continue for many years.



Self Check

1 (Fill-In) Today’s processor performance is often measured in terms of _____.

Answer: FLOPS (floating-point operations per second).

2 (Fill-In) The technology that could wreak havoc with blockchain-based cryptocurrencies, like Bitcoin, and other blockchain-based technologies is _____.

Answer: quantum computers.

3 (True/False) With cloud computing you pay a fixed price for cloud services regardless of how much you use those services?

Answer: False. A key cloud-computing benefit is that you pay for only what you use to accomplish a given task.

55. https://motherboard.vice.com/en_us/article/ywbbpm/bitcoin-mining-electricity-consumption-ethereum-energy-climate-change.

56. <https://digiconomist.net/bitcoin-energy-consumption>.

57. <https://digiconomist.net/ethereum-energy-consumption>.

58. <https://www.morganstanley.com/ideas/cryptocurrencies-global-utilities>.

59. <https://www.technologyreview.com/s/609480/bitcoin-uses-massive-amounts-of-energy-but-theres-a-plan-to-fix-it/>.

60. <http://mashable.com/2017/12/01/bitcoin-energy/>.

1.13.1 Big Data Analytics

Data analytics is a mature and well-developed academic and professional discipline. The term “data analysis” was coined in 1962,⁶¹ though people have been analyzing data using statistics for thousands of years going back to the ancient Egyptians.⁶² Big data analytics is a more recent phenomenon—the term “big data” was coined around 2000.⁶³

Consider four of the V’s of big data^{64,65}:

1. Volume—the amount of data the world is producing is growing exponentially.
2. Velocity—the speed at which that data is being produced, the speed at which it moves through organizations and the speed at which data changes are growing quickly.^{66,67,68}
3. Variety—data used to be alphanumeric (that is, consisting of alphabetic characters, digits, punctuation and some special characters)—today it also includes images, audios, videos and data from an exploding number of Internet of Things sensors in our homes, businesses, vehicles, cities and more.
4. Veracity—the validity of the data—is it complete and accurate? Can we trust that data when making crucial decisions? Is it real?

Most data is now being created digitally in a *variety* of types, in extraordinary *volumes* and moving at astonishing *velocities*. Moore’s Law and related observations have enabled us to store data economically and to process and move it faster—and all at rates growing exponentially over time. Digital data storage has become so vast in capacity, cheap and small that we can now conveniently and economically retain *all* the digital data we’re creating.⁶⁹ That’s big data.

The following Richard W. Hamming quote—although from 1962—sets the tone for the rest of this book:

*“The purpose of computing is insight, not numbers.”*⁷⁰

Data science is producing new, deeper, subtler and more valuable insights at a remarkable pace. It’s truly making a difference. Big data analytics is an integral part of the answer. We address big data infrastructure in Chapter 17 with hands-on case studies on NoSQL data-

61. <https://www.forbes.com/sites/gilpress/2013/05/28/a-very-short-history-of-data-science/>.

62. <https://www.flydata.com/blog/a-brief-history-of-data-analysis/>.

63. <https://bits.blogs.nytimes.com/2013/02/01/the-origins-of-big-data-an-etymological-detective-story/>.

64. <https://www.ibmbigdatahub.com/infographic/four-vs-big-data>.

65. There are lots of articles and papers that add many other “V-words” to this list.

66. <https://www.zdnet.com/article/volume-velocity-and-variety-understanding-the-three-vs-of-big-data/>.

67. <https://what-is.techtarget.com/definition/3Vs>.

68. <https://www.forbes.com/sites/brentdykes/2017/06/28/big-data-forget-volume-and-variety-focus-on-velocity>.

69. <http://www.lesk.com/mlesk/ksg97/ksg.html>. [The following article pointed us to this Michael Lesk article: <https://www.forbes.com/sites/gilpress/2013/05/28/a-very-short-history-of-data-science/>.]

70. Hamming, R. W., *Numerical Methods for Scientists and Engineers* (New York, NY, McGraw Hill, 1962). [The following article pointed us to Hamming’s book and his quote that we cited: <https://www.forbes.com/sites/gilpress/2013/05/28/a-very-short-history-of-data-science/>.]

bases, Hadoop MapReduce programming, Spark, real-time Internet of Things (IoT) stream programming and more.

To get a sense of big data's scope in industry, government and academia, check out the high-resolution graphic.⁷¹ You can click to zoom for easier readability:

[http://mattturck.com/wp-content/uploads/2018/07/
Matt_Turck_FirstMark_Big_Data_Landscape_2018_Final.png](http://mattturck.com/wp-content/uploads/2018/07/Matt_Turck_FirstMark_Big_Data_Landscape_2018_Final.png)

I.13.2 Data Science and Big Data Are Making a Difference: Use Cases

The data-science field is growing rapidly because it's producing significant results that are making a difference. We enumerate data-science and big data use cases in the following table. We expect that the use cases and our examples, exercises and projects will inspire interesting term projects, directed-study projects, capstone-course projects and thesis research. Big-data analytics has resulted in improved profits, better customer relations, and even sports teams winning more games and championships while spending less on players.^{72,73,74}

Data-science use cases

anomaly detection	customer churn	facial recognition
assisting people with disabilities	customer experience	fitness tracking
auto-insurance risk prediction	customer retention	fraud detection
automated closed captioning	customer satisfaction	game playing
automated image captions	customer service	genomics and healthcare
automated investing	customer service agents	Geographic Information Systems (GIS)
autonomous ships	customized diets	GPS Systems
brain mapping	cybersecurity	health outcome improvement
caller identification	data mining	hospital readmission reduction
cancer diagnosis/treatment	data visualization	human genome sequencing
carbon emissions reduction	detecting new viruses	identity-theft prevention
classifying handwriting	diagnosing breast cancer	immunotherapy
computer vision	diagnosing heart disease	insurance pricing
credit scoring	diagnostic medicine	intelligent assistants
crime: predicting locations	disaster-victim identification	Internet of Things (IoT) and
crime: predicting recidivism	drones	medical device monitoring
crime: predictive policing	dynamic driving routes	Internet of Things and weather
crime: prevention	dynamic pricing	forecasting
CRISPR gene editing	electronic health records	inventory control
crop-yield improvement	emotion detection	language translation
	energy-consumption reduction	

71. Turck, M., and J. Hao, "Great Power, Great Responsibility: The 2018 Big Data & AI Landscape," <http://mattturck.com/bigdata2018/>.

72. Sawchik, T., *Big Data Baseball: Math, Miracles, and the End of a 20-Year Losing Streak* (New York, Flat Iron Books, 2015).

73. Ayres, I., *Super Crunchers* (Bantam Books, 2007), pp. 7–10.

74. Lewis, M., *Moneyball: The Art of Winning an Unfair Game* (W. W. Norton & Company, 2004).

Data-science use cases

location-based services	predicting weather-sensitive product sales	smart thermostats
loyalty programs	predictive analytics	smart traffic control
malware detection	preventative medicine	social analytics
mapping	preventing disease outbreaks	social graph analysis
marketing	reading sign language	spam detection
marketing analytics	real-estate valuation	spatial data analysis
music generation	recommendation systems	sports recruiting and coaching
natural-language translation	reducing overbooking	stock market forecasting
new pharmaceuticals	ride sharing	student performance assessment
opioid abuse prevention	risk minimization	summarizing text
personal assistants	robo financial advisors	telemedicine
personalized medicine	security enhancements	terrorist attack prevention
personalized shopping	self-driving cars	theft prevention
phishing elimination	sentiment analysis	travel recommendations
pollution reduction	sharing economy	trend spotting
precision medicine	similarity detection	visual product search
predicting cancer survival	smart cities	voice recognition
predicting disease outbreaks	smart homes	voice search
predicting health outcomes	smart meters	weather forecasting
predicting student enrollments		

1.14 Case Study—A Big-Data Mobile Application

Google’s Waze GPS navigation app, with its 90 million monthly active users,⁷⁵ is one of the most widely used big-data apps. Early GPS navigation devices and apps relied on static maps and GPS coordinates to determine the best route to your destination. They could not adjust dynamically to changing traffic situations.

Waze processes massive amounts of **crowdsourced data**—that is, the data that’s continuously supplied by their users and their users’ devices worldwide. They analyze this data as it arrives to determine the best route to get you to your destination in the least amount of time. To accomplish this, Waze relies on your smartphone’s Internet connection. The app automatically sends location updates to their servers (assuming you allow it to). They use that data to dynamically re-route you based on current traffic conditions and to tune their maps. Users report other information, such as roadblocks, construction, obstacles, vehicles in breakdown lanes, police locations, gas prices and more. Waze then alerts other drivers in those locations.

Waze uses many technologies to provide its services. We’re not privy to how Waze is implemented, but we infer below a list of technologies they probably use. You’ll see many of these in Chapters 12–17. For example,

- Most apps created today use at least some open-source software. You’ll take advantage of many open-source libraries and tools throughout this book.

⁷⁵. <https://www.waze.com/brands/drivers/>.

- Waze communicates information over the Internet between their servers and their users' mobile devices. Today, such data typically is transmitted in JSON (JavaScript Object Notation) format, which we'll introduce in Chapter 9 and use in subsequent chapters. Often the JSON data will be hidden from you by the libraries you use.
- Waze uses speech synthesis to speak driving directions and alerts to you, and speech recognition to understand your spoken commands. We use IBM Watson's speech-synthesis and speech-recognition capabilities in Chapter 14.
- Once Waze converts a spoken natural-language command to text, it must determine the correct action to perform, which requires natural language processing (NLP). We present NLP in Chapter 12 and use it in several subsequent chapters.
- Waze displays dynamically updated visualizations such as alerts and maps. Waze also enables you to interact with the maps by moving them or zooming in and out. We create dynamic visualizations with Matplotlib and Seaborn throughout the book, and we display interactive maps with Folium in Chapters 13 and 17.
- Waze uses your phone as a streaming Internet of Things (IoT) device. Each phone is a GPS sensor that continuously streams data over the Internet to Waze. In Chapter 17, we introduce IoT and work with simulated IoT streaming sensors.
- Waze receives IoT streams from millions of phones at once. It must process, store and analyze that data immediately to update your device's maps, to display and speak relevant alerts and possibly to update your driving directions. This requires massively parallel processing capabilities implemented with clusters of computers in the cloud. In Chapter 17, we'll introduce various big-data infrastructure technologies for receiving streaming data, storing that big data in appropriate databases and processing the data with software and hardware that provide massively parallel processing capabilities.
- Waze uses artificial-intelligence capabilities to perform the data-analysis tasks that enable it to predict the best routes based on the information it receives. In Chapters 15 and 16 we use machine learning and deep learning, respectively, to analyze massive amounts of data and make predictions based on that data.
- Waze probably stores its routing information in a graph database. Such databases can efficiently calculate shortest routes. We introduce graph databases, such as Neo4J, in Chapter 17. Exercise 17.7 asks you to solve the popular "six degrees of separation" problem with Neo4j.
- Many cars are now equipped with devices that enable them to "see" cars and obstacles around them. These are used, for example, to help implement automated braking systems and are a key part of self-driving car technology. Rather than relying on users to report obstacles and stopped cars on the side of the road, navigation apps could take advantage of cameras and other sensors by using deep-learning computer-vision techniques to analyze images "on the fly" and automatically report those items. We introduce deep learning for computer vision in Chapter 16.

1.15 Intro to Data Science: Artificial Intelligence—at the Intersection of CS and Data Science

When a baby first opens its eyes, does it “see” its parent’s faces? Does it understand any notion of what a face is—or even what a simple shape is? Babies must “learn” the world around them. That’s what artificial intelligence (AI) is doing today. It’s looking at massive amounts of data and learning from it. AI is being used to play games, implement a wide range of computer-vision applications, enable self-driving cars, enable robots to learn to perform new tasks, diagnose medical conditions, translate speech to other languages in near real time, create chatbots that can respond to arbitrary questions using massive databases of knowledge, and much more. Who’d have guessed just a few years ago that artificially intelligent self-driving cars would be allowed on our roads—or even become common? Yet, this is now a highly competitive area. The ultimate goal of all this learning is **artificial general intelligence**—an AI that can perform intelligence tasks as well as humans.

Artificial-Intelligence Milestones

Several artificial-intelligence milestones, in particular, captured people’s attention and imagination, made the general public start thinking that AI is real and made businesses think about commercializing AI:

- In a 1997 match between **IBM’s DeepBlue** computer system and chess Grandmaster Gary Kasparov, DeepBlue became the first computer to beat a reigning world chess champion under tournament conditions.⁷⁶ IBM loaded DeepBlue with hundreds of thousands of grandmaster chess games.⁷⁷ DeepBlue was capable of using *brute force* to evaluate up to 200 million moves per second!⁷⁸ This is big data at work. IBM received the Carnegie Mellon University Fredkin Prize, which in 1980 offered \$100,000 to the creators of the first computer to beat a world chess champion.⁷⁹
- In 2011, **IBM’s Watson** beat the two best human Jeopardy! players in a \$1 million match. Watson simultaneously used hundreds of language-analysis techniques to locate correct answers in 200 million pages of content (including all of Wikipedia) requiring four terabytes of storage.^{80,81} Watson was trained with machine learning and **reinforcement-learning techniques**.⁸² Chapter 16 discusses **machine-learning** and Chapter 17’s exercises introduce **reinforcement learning**.
- Go—a board game created in China thousands of years ago⁸³—is widely considered to be one of the most complex games ever invented with 10^{170} possible board configurations.⁸⁴ To give you a sense of how large a number that is, it’s

76. https://en.wikipedia.org/wiki/Deep_Blue_versus_Garry_Kasparov.

77. [https://en.wikipedia.org/wiki/Deep_Blue_\(chess_computer\)](https://en.wikipedia.org/wiki/Deep_Blue_(chess_computer)).

78. [https://en.wikipedia.org/wiki/Deep_Blue_\(chess_computer\)](https://en.wikipedia.org/wiki/Deep_Blue_(chess_computer)).

79. <https://articles.latimes.com/1997/jul/30/news/mn-17696>.

80. <https://www.techrepublic.com/article/ibm-watson-the-inside-story-of-how-the-jeopardy-winning-supercomputer-was-born-and-what-it-wants-to-do-next/>.

81. [https://en.wikipedia.org/wiki/Watson_\(computer\)](https://en.wikipedia.org/wiki/Watson_(computer)).

82. <https://www.aaai.org/Magazine/Watson/watson.php>, *AI Magazine*, Fall 2010.

83. <http://www.usgo.org/brief-history-go>.

84. <https://www.pbs.org/newshour/science/google-artificial-intelligence-beats-champion-at-worlds-most-complicated-board-game>.

believed that there are (only) between 10^{78} and 10^{87} atoms in the known universe!^{85,86} In 2015, **AlphaGo**—created by Google’s DeepMind group—used *deep learning with two neural networks to beat the European Go champion Fan Hui*. Go is considered to be a far more complex game than chess. Chapter 17 discusses neural networks and deep learning.

- More recently, Google generalized its AlphaGo AI to create **AlphaZero**—a game-playing AI that *teaches itself to play other games*. In December 2017, AlphaZero learned the rules of and taught itself to play chess in less than four hours using reinforcement learning. It then beat the world champion chess program, Stockfish 8, in a 100-game match—winning or drawing every game. After *training itself* in Go for just eight hours, AlphaZero was able to play Go vs. its AlphaGo predecessor, winning 60 of 100 games.⁸⁷ Chapter 17 discusses reinforcement learning.

A Personal Anecdote

When one of the authors, Harvey Deitel, was an undergraduate student at MIT in the mid-1960s, he took a graduate-level artificial-intelligence course with Marvin Minsky (to whom this book is dedicated), one of the founders of artificial intelligence (AI). Harvey:

Professor Minsky required a major term project. He told us to think about what intelligence is and to make a computer do something intelligent. Our grade in the course would be almost solely dependent on the project. No pressure!

I researched the standardized IQ tests that schools administer to help evaluate their students’ intelligence capabilities. Being a mathematician at heart, I decided to tackle the popular IQ-test problem of predicting the next number in a sequence of numbers of arbitrary length and complexity. I used interactive Lisp running on an early Digital Equipment Corporation PDP-1 and was able to get my sequence predictor running on some pretty complex stuff, handling challenges well beyond what I recalled seeing on IQ tests. Lisp’s ability to manipulate arbitrarily long lists recursively was exactly what I needed to meet the project’s requirements. Python offers recursion (Chapter 11) and generalized list processing (Chapter 5).

I tried the sequence predictor on many of my MIT classmates. They would make up number sequences and type them into my predictor. The PDP-1 would “think” for a while—often a long while—and almost always came up with the right answer.

Then I hit a snag. One of my classmates typed in the sequence 14, 23, 34 and 42. My predictor went to work on it, and the PDP-1 chugged away for a long time, failing to predict the next number. I couldn’t get it either. My classmate told me to think about it overnight, and he’d reveal the answer the next day, claiming that it was a simple sequence. My efforts were to no avail.

The following day he told me the next number was 57, but I didn’t understand why. So he told me to think about it overnight again, and the following day he

85. <https://www.universetoday.com/36302/atoms-in-the-universe/>.

86. https://en.wikipedia.org/wiki/Observable_universe#Matter_content.

87. <https://www.theguardian.com/technology/2017/dec/07/alphazero-google-deepmind-ai-beats-champion-program-teaching-itself-to-play-four-hours>.

said the next number was 125. That didn't help a bit—I was stumped. He said that the sequence was the numbers of the two-way crosstown streets of Manhattan. I cried, “foul,” but he said it met my criterion of predicting the next number in a numerical sequence. My world view was mathematics—his was broader.

Over the years, I've tried that sequence on friends, relatives and professional colleagues. A few who either lived in Manhattan or spent time there got it right. My sequence predictor needed a lot more than just mathematical knowledge to handle problems like this, requiring (a possibly vast) world knowledge.

Watson and Big Data Open New Possibilities

When Paul and I started working on this Python book, we were immediately drawn to IBM's Watson using big data and artificial-intelligence techniques like natural language processing (NLP) and machine learning to beat two of the world's best human Jeopardy! players. We realized that Watson could probably handle problems like the sequence predictor because it was loaded with the world's street maps and a whole lot more. That whet our appetite for digging in deep on big data and today's artificial-intelligence technologies.

It's notable that all of the data-science implementation case studies in Chapters 12 to 17 either are rooted in artificial intelligence technologies or discuss the big data hardware and software infrastructure that enables data scientists to implement leading-edge AI-based solutions effectively.

AI: A Field with Problems But No Solutions

For many decades, AI has been a field with problems and *no* solutions. That's because once a particular problem is solved people say, “Well, that's not intelligence, it's just a computer program that tells the computer exactly what to do.” However, with machine learning (Chapter 15), deep learning (Chapter 16) and reinforcement learning (Chapter 16 exercises), we're not pre-programming solutions to *specific* problems. Instead, we're letting our computers solve problems by learning from data—and, typically, lots of it.

Many of the most interesting and challenging problems are being pursued with deep learning. Google alone has thousands of deep-learning projects underway and that number is growing quickly.^{88, 89} As you work through this book, we'll introduce you to many edge-of-the-practice artificial intelligence, big data and cloud technologies and you'll work through hundreds of (often intriguing) examples, exercises and projects.



Self Check

1 (Fill-In) The ultimate goal of AI is to produce a(n) _____.

Answer: artificial general intelligence.

2 (Fill-In) IBM's Watson beat the two best human Jeopardy! players. Watson was trained using a combination of _____ learning and _____ learning techniques.

Answer: machine, reinforcement.

88. <http://theweek.com/speedreads/654463/google-more-than-1000-artificial-intelligence-projects-works>.

89. <https://www.zdnet.com/article/google-says-exponential-growth-of-ai-is-changing-nature-of-compute/>.

3 (Fill-In) Google's _____ taught itself to play chess in less than four hours using reinforcement learning, then beat the world champion chess program, Stockfish 8, in a 100-game match—winning or drawing every game.

Answer: AlphaZero.

Exercises

1.1 (IPython Session) Using the techniques you learned in Section 1.10.1, execute the following expressions. Which, if any, produce a runtime error?

- a) `10 / 3`
- b) `10 // 3`
- c) `10 / 0`
- d) `10 // 0`
- e) `0 / 10`
- f) `0 // 10`

1.2 (IPython Session) Using the techniques you learned in Section 1.10.1, execute the following expressions. Which, if any, produce a runtime error?

- a) `10 / 3 + 7`
- b) `10 // 3 + 7`
- c) `10 / (3 + 7)`
- d) `10 / 3 - 3`
- e) `10 / (3 - 3)`
- f) `10 // (3 - 3)`

1.3 (Creating a Jupyter Notebook) Using the techniques you learned in Section 1.10.3, create a Jupyter Notebook containing cells for the previous exercise's expressions and execute those expressions.

1.4 (Computer Organization) Fill in the blanks in each of the following statements:

- a) The logical unit that receives information from outside the computer for use by the computer is the _____.
- b) _____ is a logical unit that sends information which has already been processed by the computer to various devices so that it may be used outside the computer.
- c) _____ and _____ are logical units of the computer that retain information.
- d) _____ is a logical unit of the computer that performs calculations.
- e) _____ is a logical unit of the computer that makes logical decisions.
- f) _____ is a logical unit of the computer that coordinates the activities of all the other logical units.

1.5 (Clock as an Object) Clocks are among the world's most common objects. Discuss how each of the following terms and concepts applies to the notion of a clock: class, object, instantiation, instance variable, reuse, method, inheritance (consider, for example, an alarm clock), superclass, subclass.

1.6 (Gender Neutrality) Write down the steps of a manual procedure for processing a paragraph of text and replacing gender-specific words with gender-neutral ones. Assuming that you've been given a list of gender-specific words and their gender-neutral replacements (for example, replace "wife" or "husband" with "spouse," replace "man" or "woman" with "person," replace "daughter" or "son" with "child," and so on), explain the

procedure you'd use to read through a paragraph of text and manually perform these replacements. How might your procedure generate a strange term like "woperchild" and how might you modify your procedure to avoid this possibility? In Chapter 3, you'll learn that a more formal computing term for "procedure" is "algorithm," and that an algorithm specifies the *steps* to be performed and the *order* in which to perform them.

1.7 (Self-Driving Cars) Just a few years back the notion of driverless cars on our streets would have seemed impossible (in fact, our spell-checking software doesn't recognize the word "driverless"). Many of the technologies you'll study in this book are making self-driving cars possible. They're already common in some areas.

- a) If you hailed a taxi and a driverless taxi stopped for you, would you get into the back seat? Would you feel comfortable telling it where you want to go and trusting that it would get you there? What kinds of safety measures would you want in place? What would you do if the car headed off in the wrong direction?
- b) What if two self-driving cars approached a one-lane bridge from opposite directions? What protocol should they go through to determine which car should proceed?
- c) If a police officer pulls over a speeding self-driving car in which you're the only passenger, who—or what entity—should pay the ticket?
- d) What if you're behind a car stopped at a red light, the light turns green and the car doesn't move? You honk and nothing happens. You get out of your car and notice that there's no driver. What would you do?
- e) One serious concern with self-driving vehicles is that they could potentially be hacked. Someone could set the speed high (or low), which could be dangerous. What if they redirect you to a destination other than what you want?
- f) Imagine other scenarios that self-driving cars will encounter.

1.8 (Research: Reproducibility) A crucial concept in data-science studies is reproducibility, which helps others (and you) reproduce your results. Research reproducibility and list the concepts used to create reproducible results in data-science studies. Research and discuss the part that Jupyter Notebooks play in reproducibility.

1.9 (Research: Artificial General Intelligence) One of the most ambitious goals in the field of AI is to achieve *artificial general intelligence*—the point at which machine intelligence would equal human intelligence. Research this intriguing topic. When is this forecast to happen? What are some key ethical issues this raises? Human intelligence seems to be stable over long periods. Powerful computers with artificial general intelligence could conceivably (and quickly) evolve intelligence far beyond that of humans. Research and discuss the issues this raises.

1.10 (Research: Intelligent Assistants) Many companies now offer computerized intelligent assistants, such as IBM Watson, Amazon Alexa, Apple Siri, Google Assistant and Microsoft Cortana. Research these and others and list uses that can improve people's lives. Research privacy and ethics issues for intelligent assistants. Locate amusing intelligent-assistant anecdotes.

1.11 (Research: AI in Health Care) Research the rapidly growing field of AI big-data applications in health care. For example, suppose a diagnostic medical application had access to every x-ray that's ever been taken and the associated diagnoses—that's surely big data. As you'll see in the "Deep Learning" chapter, computer-vision applications can work with

this "labeled" data to learn to diagnose medical problems. Research deep learning in diagnostic medicine and describe some of its most significant accomplishments. What are some ethical issues of having machines instead of human doctors performing medical diagnoses? Would you trust a machine-generated diagnosis? Would you ask for a second opinion?

1.12 (*Research: Big Data, AI and the Cloud—How Companies Use These Technologies*)

For a major organization of your choice, research how they may be using each of the following technologies that you'll use in this book: Python, AI, big data, the cloud, mobile, natural language processing, speech recognition, speech synthesis, database, machine learning, deep learning, reinforcement learning, Hadoop, Spark, Internet of Things (IoT) and web services.

1.13 (*Research: Raspberry Pi and the Internet of Things*)

It's now possible to have a computer at the heart of just about any type of device and to connect those devices to the Internet. This has led to the Internet of Things (IoT), which already interconnects tens of billions of devices. The Raspberry Pi is an economical computer which is often at the heart of IoT devices. Research the Raspberry Pi and some of the many IoT applications in which it's used.

1.14 (*Research: The Ethics of Deep Fakes*)

Artificial-intelligence technologies are making it possible to create *deep fakes*—realistic fake videos of people that capture their appearance, voice, body motions and facial expressions. You can have them say and do whatever you specify. Research the ethics of deep fakes. What would happen if you turned on your TV and saw a deep-fake video of a prominent government official or newscaster reporting that a nuclear attack was about to happen? Research Orson Welles and his "War of the Worlds" radio broadcast of 1938, which created mass panic.

1.15 (*Public-Key Cryptography*)

Cryptography is a crucial technology for privacy and security. Research Python's cryptography capabilities. Research online for a simple explanation of how public-key cryptography is used to implement the BitCoin cryptocurrency.

1.16 (*Blockchain: A World of Opportunity*)

Cryptocurrencies like Bitcoin and Ethereum are based on a technology called blockchain that has seen explosive growth over the last few years. Research blockchain's origin, applications and how it came to be used as the basis for cryptocurrencies. Research other major applications of blockchain. Over the next many years there will be extraordinary opportunities for software developers who thoroughly understand blockchain applications development.

1.17 (*OWASP Python Security Project*)

Building secure computer applications is a tremendous challenge. Many of the world's largest companies, government agencies, and military organizations have had their systems compromised. The OWASP project is concerned with "hardening" computer systems and applications to resist attacks. Research OWASP and discuss their accomplishments and current challenges.

1.18 (*IBM Watson*)

We discuss IBM's Watson in Chapter 14. You'll use its cognitive computing capabilities to quickly build some intriguing applications. IBM is partnering with tens of thousands of companies—including our publisher, Pearson Education—across a wide range of industries. Research some of Watson's key accomplishments and the kinds of challenges IBM and its partners are addressing.

1.19 (*Research: Mobile App Development with Python*)

Research the tools that are available for Python-based iOS and Android app development, such as BeeWare, Kivy, Py-

Mob, Pythonista and others. Which of these are cross-platform? Mobile applications development is one of the fastest growing areas of software development, and it's a great source of class projects, directed study projects, capstone exercise projects and even thesis projects. With cross-platform app-development tools, you'll be able to write your own apps and deploy them on many app stores quickly.